LangGraph를 활용한 AI Agent 개발하기

강사 : 오영제

과정 구성

Part A — 고객지원 에이전트 한 개 완성하기

01 LangChain과 LangGraph 비교하기
02 LangGraph의 핵심 요소 이해하기
03 Simple Graph 작성 및 실행하기
04 Retrieval Agent 작성하기
05 답변을 검증하는 Self-RAG 구축하기
06 LangChain과 LangGraph에서 도구(Tool) 활용하기
07 Function Calling과 Tool Use 패턴 적용하기
08 Checkpoint와 영속성(Persistence) 구현하기

Part B — 팀 · 사람 · 외부 시스템과 연결하기

09 Multi-Agent의 개념과 장점 이해하기
10 다중 에이전트로 문제 해결하기
11 사용자의 피드백을 수용하는 Node 구현하기
12 MCP의 필요성과 아키텍처 이해하기
13 MCP 서버 개발하기
14 MCP Client 활용하기
15 LangGraph 상태와 MCP Context 동기화하기
16 LangGraph Agent 배포하기

실습 환경 설정

Node.js · VS Code 설치

Node.js (런타임)

- https://nodejs.org 에서 LTS(24.x) 설치
- npm(패키지 관리자)이 함께 설치된다
- 22.13 이상이면 실습 가능 (node:sqlite 내장)

VS Code (편집기)

- https://code.visualstudio.com 설치
- F5 → '현재 실습 파일 실행'으로 TS 파일 바로 실행

설치 확인 (터미널)

```
node -v          # v24.x.x
npm -v           # 11.x.x
code --version   # VS Code 설치 확인 (터미널에서)
```

실습 프로젝트 열기와 패키지 설치

터미널

```
# 1) 실습 프로젝트 폴더로 이동 → VS Code로 열기
cd langgraph-ts-labs
code .

# 2) 패키지 설치 (package.json 기준)
npm install

# 3) 환경 변수 파일 만들기
copy .env.example .env

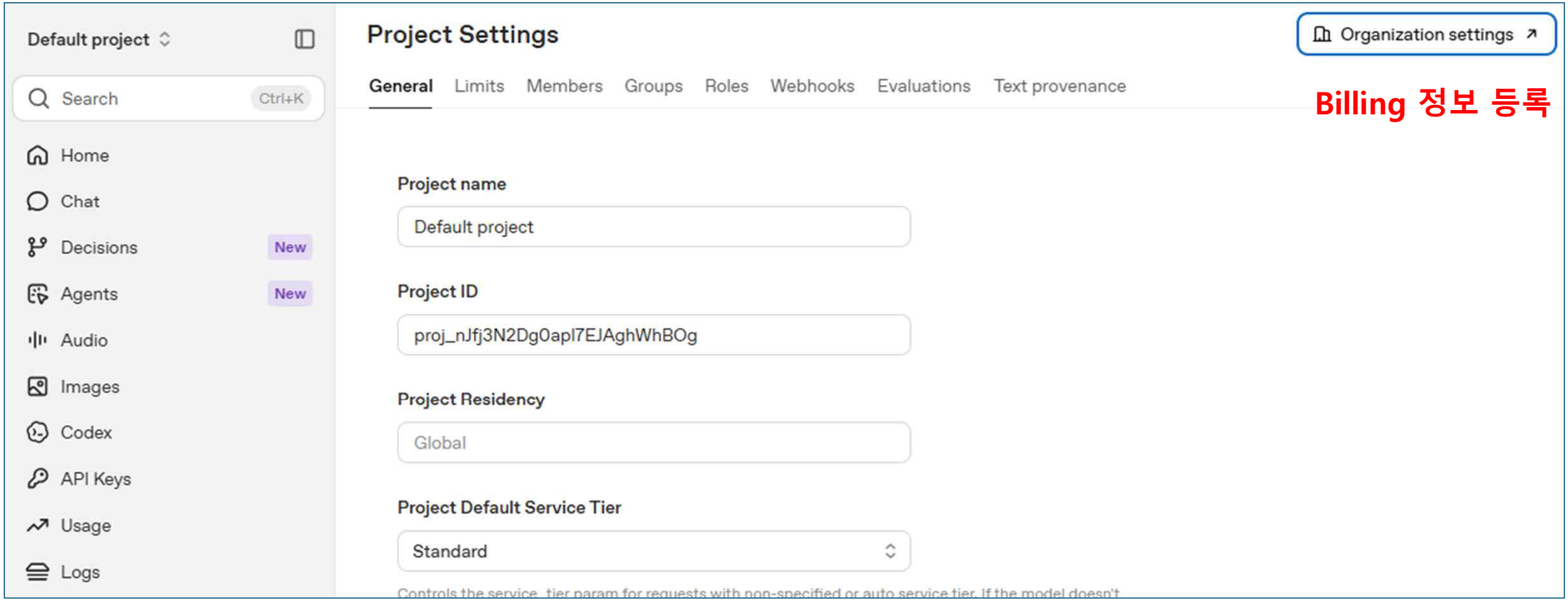
# 4) 첫 실습 실행해 보기
npx tsx labs/00_first_agent/solution.ts
```

명령+	용도
npx tsx <파일>	TypeScript 파일 실행
npm run download-model	KURE 로컬 임베딩 받기 (OpenAI 키가 없을 때)
npm run make-exercises	(강사용) solution.ts → exercise.ts 생성
npm run typecheck	타입 검사
npm run check-keys	API 키 검사
npm run dev	Agent Server + Studio

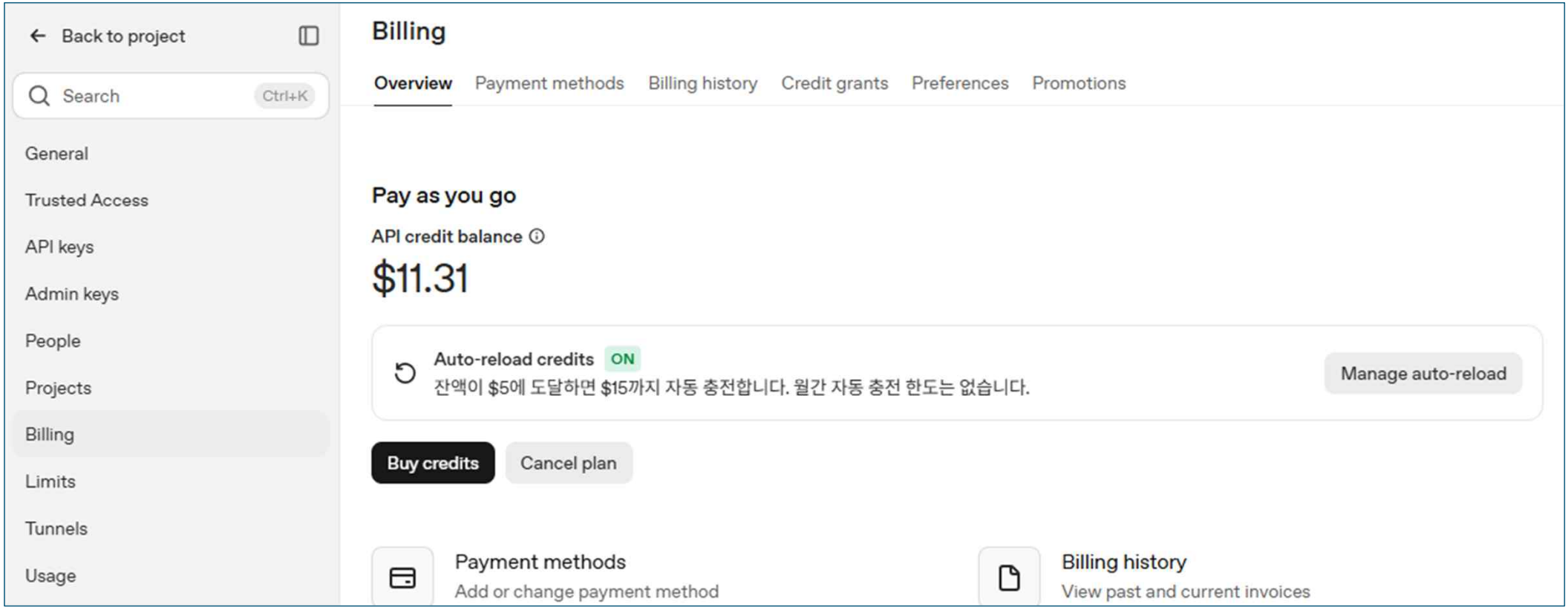
주요 패키지

- @langchain/langgraph · langchain · @langchain/core
- @langchain/openai · anthropic · google
- @langchain/mcp-adapters · @modelcontextprotocol/*
- zod · tsx

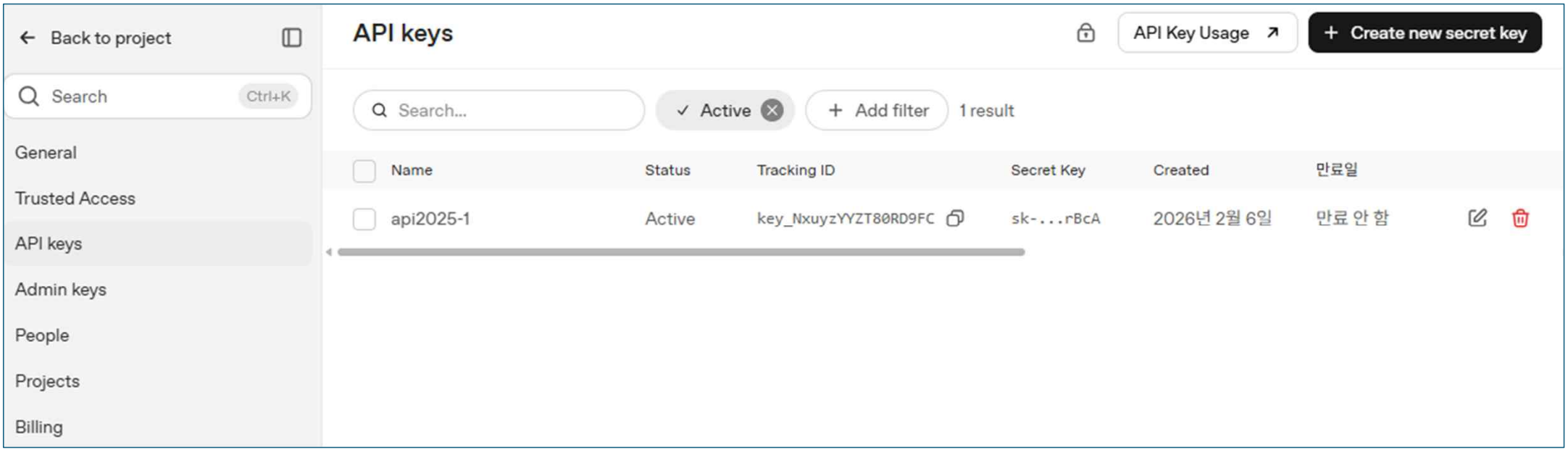
OpenAI API Key 생성 : https://openai.com/index/openai-api/



Billing 정보 등록



API key 생성

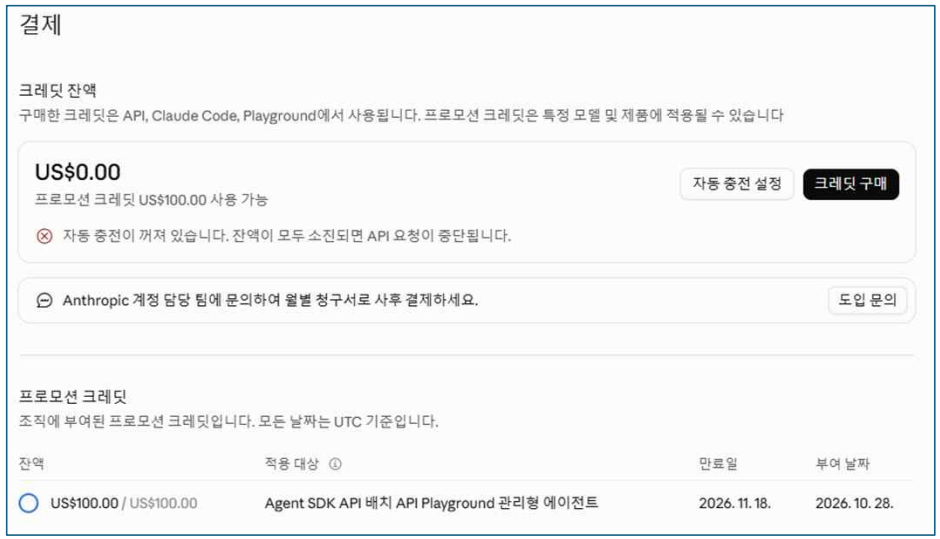

Back to project
Search
Ctrl+K
General
Trusted Access
API keys
Admin keys
People
Projects
Billing
API keys
API Key Usage
Create new secret key
Search...
Active
Add filter
1 result
Name
Status
Tracking ID
Secret Key
Created
만료일
api2025-1
Active
key_NxuyzYYZT80RD9FC
sk-...rBcA
2026년 2월 6일
만료 안 함

Gemini API KEY 생성



Anthropic API KEY 생성





프로모션 크레딧

.env 설정과 모델 선택 — OpenAI · Claude · Gemini

.env

```
# .env  (git에 올리지 않는다 → .gitignore)
# 사용하는 회사의 키만 넣는다
OPENAI_API_KEY=sk-...
ANTHROPIC_API_KEY=
GOOGLE_API_KEY=
# 선택: Studio · 실행 추적
LANGSMITH_API_KEY=
```

모델은 실습 파일 맨 위 MODEL로 고른다

```
// labs/00_first_agent/solution.ts
import "dotenv/config";   // .env의 키를 읽는다

const MODEL = "openai:gpt-6-luna";
//   "anthropic:claude-haiku-4-5"
//   "google:gemini-3.6-flash"

createAgent({ model: MODEL, tools, systemPrompt });
```

제공자	키 발급	MODEL 값 예
OpenAI	platform.openai.com/api-keys	openai:gpt-5.4-mini
Anthropic	console.anthropic.com	anthropic:claude-haiku-4-5
Google	aistudio.google.com	google:gemini-3.6-flash

셋 중 하나의 키만 있으면 된다. 임베딩은 OpenAI 키가 있으면 text-embedding-3-small, 없으면 KURE-v1 로컬 한국어 임베딩(npm run download-model)을 쓴다.

LangChain과 LangGraph 비교하기

Module 01 | 프레임워크·런타임의 역할 구분

LangChain이란?

- 대규모 언어 모델(LLM)을 활용한 애플리케이션 개발 프레임워크
- LLM 애플리케이션의 전체 라이프사이클을 간소화
- Python 및 TypeScript 지원
- **LangChain** - 원하는 모델 제공자를 사용해 에이전트를 빠르고 손쉽게 구축
- **LangGraph** - 에이전트의 모든 단계를 세밀하게 제어하는 저수준 오케스트레이션, 메모리, Human-in-the-loop

구분	LangChain	LangGraph
성격	오픈소스 라이브러리	오픈소스 라이브러리
주요 목적	LLM 애플리케이션 개발을 쉽고 빠르게	에이전트 워크플로우를 세밀하게 제어
핵심 기능	모델 연동, 프롬프트, 체인, 검색(RAG), 도구 연동 등	그래프 기반 오케스트레이션, 메모리, 상태 관리, Human-in-the-loop
실행 방식	개발자가 직접 구축/실행	개발자가 직접 구축/실행
제공 범위	빌딩 블록(컴포넌트)	워크플로우 엔진
대표 사용 사례	RAG 챗봇, 문서 요약, 도구 연동 애플리케이션	복잡한 멀티스텝 에이전트, 승인 워크플로우, 상태 관리가 필요한 서비스

LangChain · LangGraph — 2계층 구조

LangChain

Framework

createAgent · 미들웨어 · 모델/도구 표준 인터페이스

LangGraph

Runtime

State 그래프 · 지속 실행 · 스트리밍 · 체크포인트 · interrupt

위 계층은 아래 계층 위에서 동작한다 — 결과물은 모두 LangGraph 그래프

언제 무엇을 쓰나

- LangChain: 표준 도구 호출 에이전트를 빠르게 구축
- LangGraph: 에이전트에 검증 · 승인 · 기억 · 팀 구성을 직접 설계해 넣을 때

출처: LangChain 공식 문서 — Frameworks, runtimes, and harnesses

이 과정의 흐름

스마트몰 고객지원 에이전트를 LangChain createAgent로 시작해(Module 01), LangGraph로 직접 설계하며 키웁니다(Module 02~16).

LangChain 주요 개념 (Core Components)

1. **Agents (에이전트)** - 사용자 입력에 따라 도구를 선택하고 실행하는 자율적 시스템
2. **Models (모델)** - 채팅에 특화된 LLM (Chat Models), 다양한 제공자 통합 지원
3. **Messages (메시지)** - 채팅 모델의 입력과 출력 단위 (Human, AI, System, Tool)
4. **Tools (도구)** - 모델이 호출할 함수 정의 및 Tool Calling 기능
5. **Short-term memory (단기 메모리)** - 대화 기록을 메시지 시퀀스로 관리
6. **Streaming (스트리밍)** - 모델 응답을 토큰 단위로 실시간 전달
7. **Structured output (구조화된 출력)** - JSON 등 지정 스키마에 맞춰 응답

LangChain Messages

메시지 구성 요소

Role (역할) → **메시지의 유형** 식별,

Content (내용) → 메시지의 실제 내용 (텍스트, 이미지, 오디오 등)

Metadata (메타데이터) → 선택적(optional) 필드. 응답 정보(response info), 메시지 ID, 토큰 사용량(token usage) 등

메시지 유형 (Message types)

System message (시스템 메시지)

Human message (사용자 메시지)

AI message (AI 메시지)

Tool message (도구 메시지)

OpenAI Chat Completions API (원형)

```javascript
import OpenAI from "openai";
const client = new OpenAI();
const r = await client.chat.completions.create({
  model: "gpt-5.4-mini",
  messages: [
    { role: "system", content: "너는 친절한 튜터야" },
    { role: "user", content: "RAG를 3줄로 설명해줘" },
  ],
});
console.log(r.choices[0].message.content);
```

LangChain · 메시지 넣는 두 가지 방법

```javascript
import { SystemMessage, HumanMessage }
  from "langchain";
// ① 메시지 클래스
await model.invoke([
  new SystemMessage("너는 친절한 튜터야"),
  new HumanMessage("RAG를 3줄로 설명해줘"),
]);
// ② role · content 객체 ("user" = HumanMessage)
const resp = await model.invoke([
  { role: "system", content: "너는 친절한 튜터야" },
  { role: "user", content: "RAG를 3줄로 설명해줘" },
]);
console.log(resp.text);
```

함수 호출 단계 — Developer와 Model의 왕복

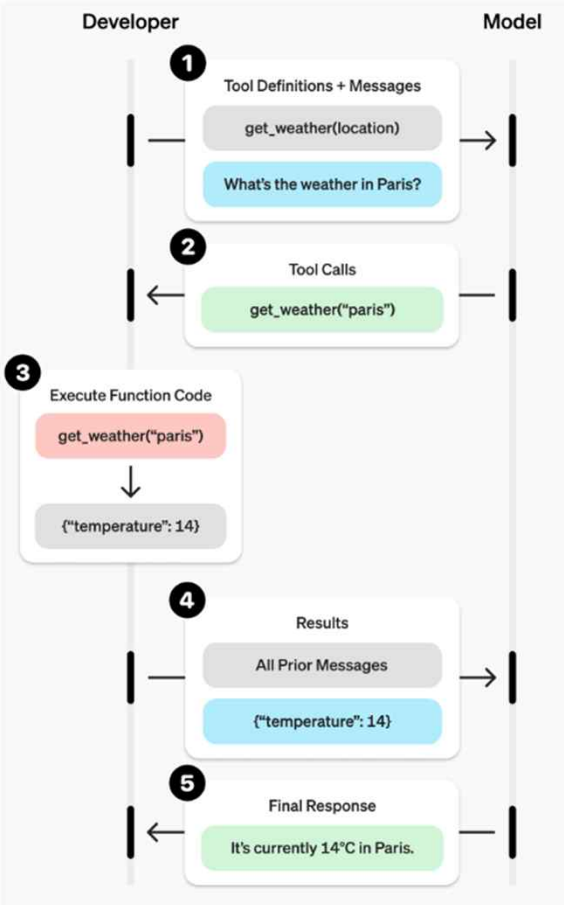


그림: OpenAI Function Calling 문서

함수 호출 단계와 LangChain 코드

1 **함수 정보와 함께 LLM 모델 호출**

Developer → Model · 도구 정의 + 메시지 · **model.bindTools(tools)**

2 **모델이 함수 호출을 결정 – 함수명과 argument 반환**

Model → Developer · tool_calls · **ai.tool_calls { name, args, id }**

3 **함수 호출 – 모델이 만들어준 argument 이용**

Developer · 함수 코드 실행 · **ToolNode 실행 (createAgent는 자동)**

4 **함수 호출 결과를 모델에 제공**

Developer → Model · 이전 메시지 + 실행 결과 · **ToolMessage { tool_call_id }**

5 **모델이 최종 응답 생성**

Model → Developer · 자연어 답변 · **마지막 AIMessage.text**

Agents

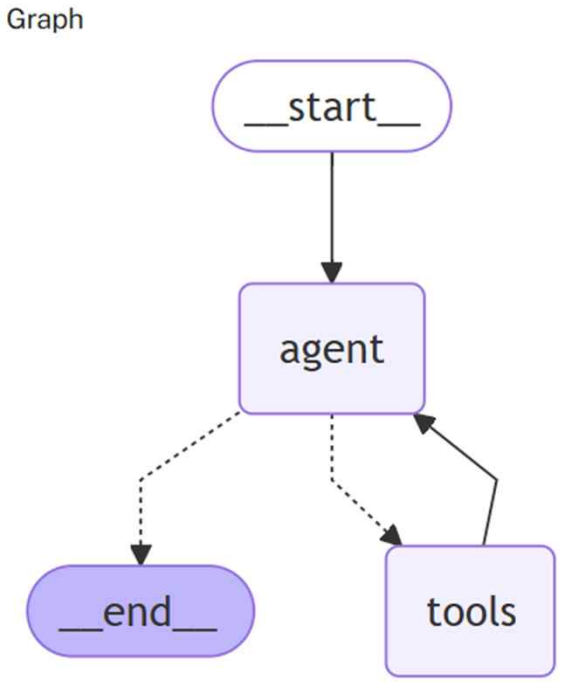
Agent 란 ?

- LLM을 핵심 엔진으로 사용하여 주어진 목표를 달성하기 위해 독립적으로 작업(추론, 판단, 실행, 피드백)을 수행하는 인공지능 시스템

- 주로 LLM(대규모 언어 모델)의 능력을 기반으로 동작하며, 사용자의 명령을 이해하고, 판단하며, 실행

- 즉, LLM은 두뇌, Agent는 이 두뇌를 활용해 행동을 실행하는 작업자 역할

- 활용 사례
  - 고객 지원 챗봇: 전자상거래 플랫폼에서 환불 요청을 처리하거나 배송 상태 확인
  - 연구 도우미: 과학 논문에서 특정 주제에 대한 핵심 내용 추출
  - 개인 비서: 이메일 초안을 작성하고, 미팅 일정 조율
  - 데이터 분석 에이전트: 주식 시장 데이터를 분석해 투자 전략 제안
  - 교육용 튜터: 수학 문제 풀이 과정을 단계별로 설명



Agent 생성 & 상태 관리

createAgent()

- model + tools → 즉시 실행 가능한 에이전트
- system prompt로 에이전트 역할 지정
- invoke() / stream() 두 가지 실행 방식

에이전트 상태(AgentState)

- messages: 대화 기록 자동 누적
- thread_id로 대화 세션 분리
- checkpointer 연결 시 상태 영속화

Agent 실행 흐름

- 1. 사용자 입력 수신
- 2. LLM 추론 (Reason)
- 3. 도구 호출 (Action)
- 4. 결과 관찰 (Observe)
- 5. 완료 판단 → 응답 반환

상태 관리

- config = { configurable: { thread_id: "abc" } }
- getState()로 현재 상태 조회

도구 정의 – tool() 함수

tool(함수, 설정)

- 일반 TypeScript 함수를 에이전트용 도구로 변환
- name → LLM 이 인식하는 도구명
- description → 모델이 도구 목적을 이해하는 설명
- schema → 함수 호출 인수의 형식 ·
  zod 스키마(schema)로 타입 자동 생성

필드 제약 · 설명

- z.enum([...]) → 정해진 값 중 하나만
- .min() · .max() · .int() → 범위 · 길이 제한
- .describe() → 필드 의미를 모델에게 전달

```
import { tool } from "langchain";
import * as z from "zod";

const getOrderStatus = tool(
  async ({ orderId }) => {
    const o = findOrder(orderId);
    return o ? `${o.id}: ${o.status}`
             : "주문을 찾을 수 없습니다.";
  },
  {
    name: "get_order_status",
    description: "주문번호로 주문 상태를 조회한다.",
    schema: z.object({ orderId: z.string() }),
  },
);

// 필드 제약 예 – create_ticket 입력
z.object({
  category: z.enum(["배송", "환불", "결제", "기타"]),
  priority: z.number().int().min(1).max(3)
    .describe("1: 긴급 ~ 3: 일반"),
})
```

함수

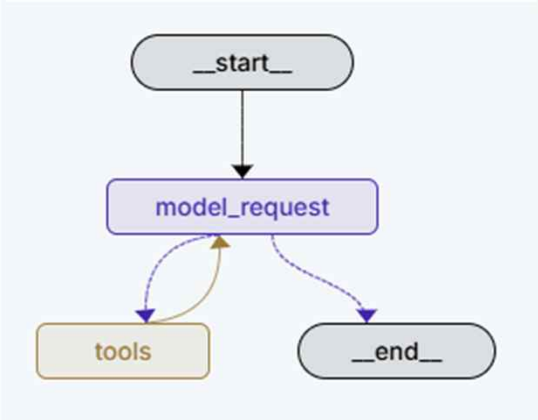
설정

실습 : labs/00_first_agent

Example

실습 목표

- 주문 조회 도구 get_order_status를 tool() + zod로 정의
- createAgent로 고객지원 에이전트 v0 만들기
- 메시지 흐름(human → ai → tool → ai) 확인



실행하기

1. 예제 열기: labs/00_first_agent/solution.ts
2. tool() + zod로 만든 get_order_status 읽기
3. createAgent(model · tools · systemPrompt)
4. npx tsx labs/00_first_agent/solution.ts
5. 주문 질문 vs 인사말 메시지 흐름 비교

주문 데이터: src/data/orders.ts (고객 3명 · 주문 10건, 가상 데이터)

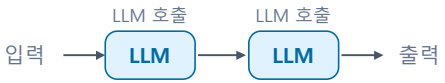
LangGraph의 핵심 요소 이해하기

Module 02 | 에이전트를 State · Node · Edge로 설계하기

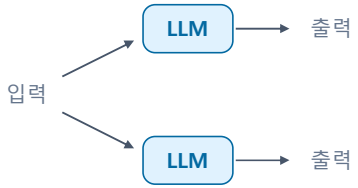
Workflows vs Agents

워크플로 (Workflows)

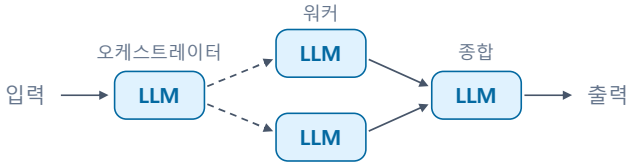
프롬프트 체이닝



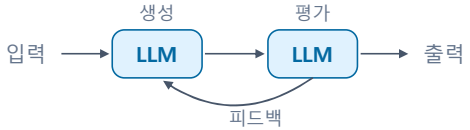
병렬화



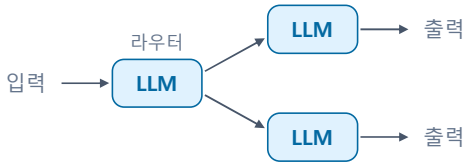
오케스트레이터-워커



평가자-최적화



라우팅

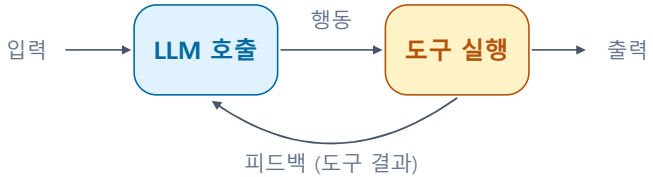


LLM이 미리 정해진
코드 경로 안에서 동작한다

LLM이 미리 정해진 코드 경로 위에서
흐름(다음 단계)을 고른다

✓ 경로를 코드가 미리 정한다 → 예측 · 테스트가 쉽다 → 단계가 분명한 업무

에이전트 (Agent)



LLM이 환경 피드백을 보고
스스로 다음 행동을 정한다

✓ 실행마다 경로가 다르다 → 열린 문제 (고객 상담)

그림 출처: LangChain Docs — Workflows and agents (한국어로 다시 그림)

3가지 핵심 구성 요소 — State, Node, Edge

State — 공유 데이터 구조

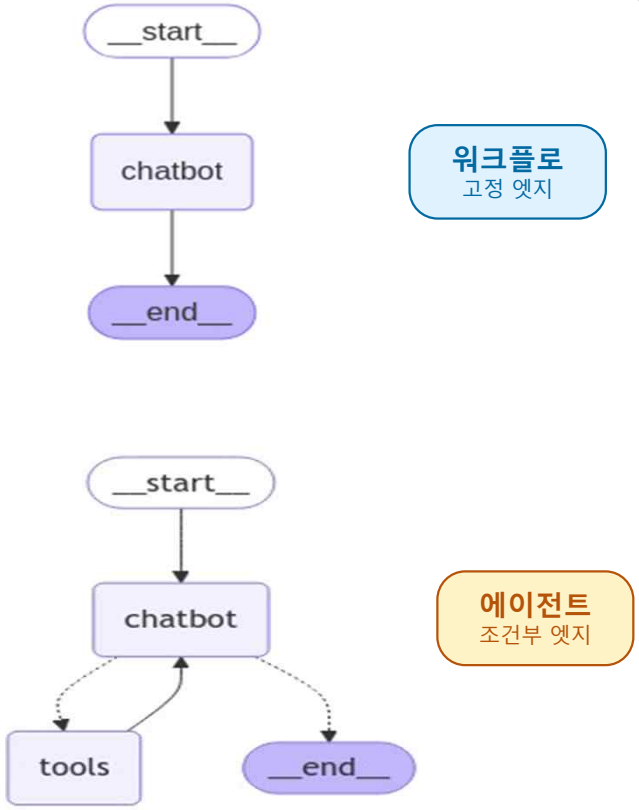
- 애플리케이션의 현재 스냅샷을 나타내는 공유 데이터
- 노드 간 전달되며, 각 노드가 읽고 업데이트
- StateSchema + zod 스키마로 정의

Node — 작업의 단위

- 에이전트 로직을 인코딩하는 TypeScript 함수 (GraphNode)
- 현재 상태를 입력받아 계산 수행 후 업데이트된 상태 반환
- LLM 호출 또는 일반 코드 모두 가능

Edge — 흐름 제어

- 현재 상태에 따라 다음 실행할 노드를 결정하는 함수
- **고정 엣지**(직접 연결) 또는 **조건부 엣지**(동적 분기)



State — 그래프가 공유하는 데이터

Core

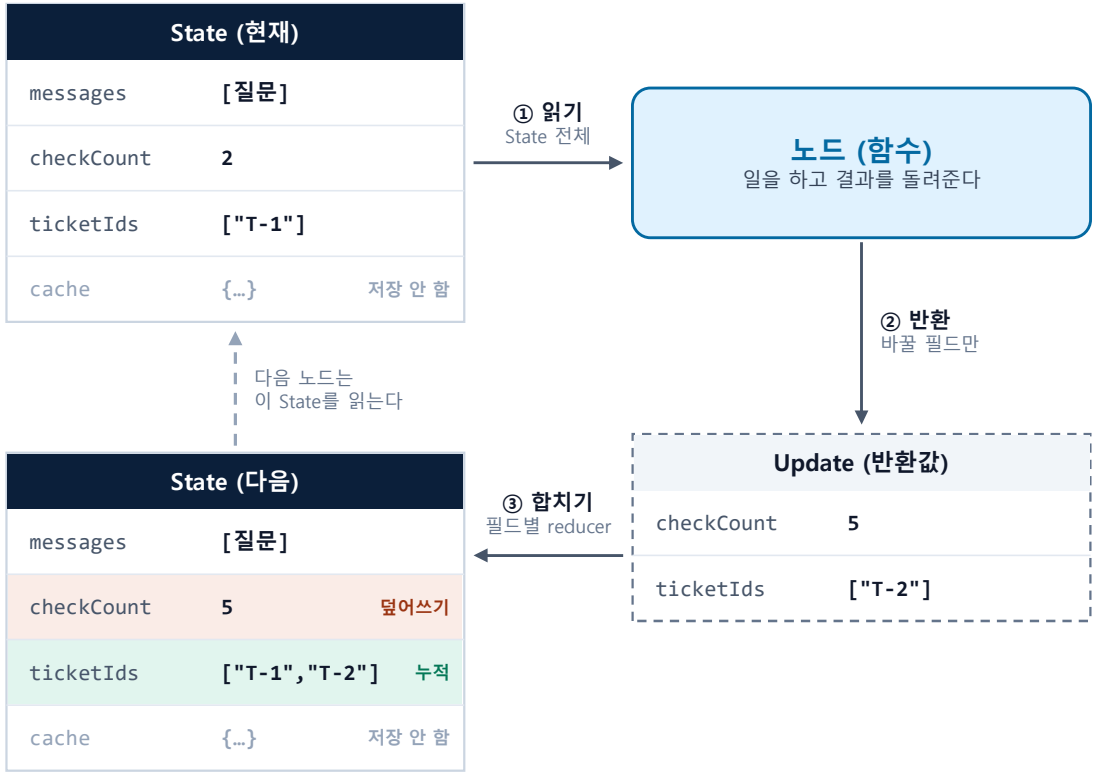
State란?

- 그래프 전체가 **공유하는 데이터**
- 노드는 읽고, **바꿀 부분만 반환** (Update)
- 필드별 **reducer**로 기존 값과 합친다

StateSchema로 필드 정의

- MessagesValue → 메시지 누적 (id 기준)
- reducer 미지정 → 덮어쓰기 (checkCount)
- ReducedValue → reducer 지정 (ticketIds)
- UntrackedValue → 저장하지 않는 값 (cache)

State가 흐르는 방식 — 노드 하나가 실행될 때



State 업데이트 규칙 — 덮어쓰기 vs 누적 (Reducer)

Core



reducer가 없는 필드에 같은 단계의 노드 둘이 값을 쓰면 → **InvalidUpdateError**

Node — 작업의 단위

노드 함수 설계 원칙

- "각 노드는 한 가지 일을 잘 수행"하도록 설계
- State를 입력받아 부분 업데이트를 반환
- LLM 호출, API 연동, 데이터 변환 등 자유
- START 노드: 그래프 실행 시작점
- END 노드: 실행 종료를 나타내는 특수 노드

노드 분리의 이점

- 디버깅과 관찰 가능성(Observability) 향상
- 각 노드를 독립적으로 테스트 가능
- 노드 수가 많아도 성능 저하 없음
- 체크포인트가 비동기로 백그라운드 저장
- 캐싱은 노드 함수 내에서 직접 구현

노드 = State를 받아 Update를 돌려주는 함수



그래프에 등록하고 실행

```
graph.addNode("addOne", addOne)
     .addEdge(START, "addOne")
     .addEdge("addOne", END);
invoke({ count: 0 })  // → { count: 1 }
```

Edge — 흐름 제어

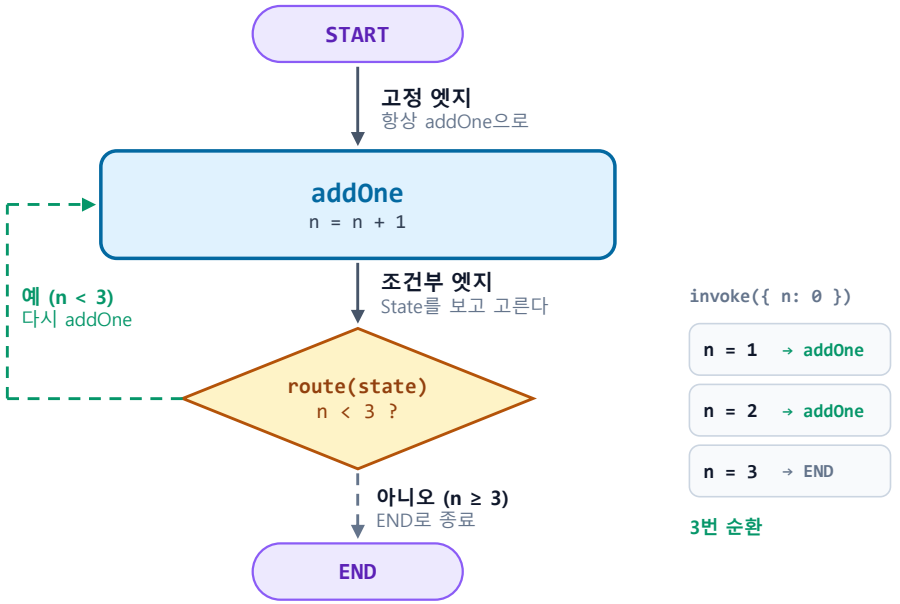
엣지 유형

- 고정 엣지(Normal): 무조건 다음 노드로 이동
- 조건부 엣지(Conditional): 상태에 따라 분기
- 조건부 진입점: 시작부터 동적 라우팅
- Send: 동적으로 노드에 데이터 전송
- Command: 상태 업데이트 + 다음 노드를 함께 지정

조건부 엣지 패턴

- addConditionalEdges()로 분기 로직 등록
- 라우팅 함수가 State를 받아 다음 노드명 반환
- 여러 노드 중 하나로 분기 가능
- 순환(Cycle) 구현의 핵심 메커니즘
- ReAct 패턴(추론→행동 반복)의 기반

엣지 = 다음에 실행할 노드를 정하는 연결




```
const route = (s) => (s.n < 3 ? "addOne" : END);
graph.addEdge(START, "addOne")          // 고정
     .addConditionalEdges("addOne", route,
       ["addOne", END]);                // 조건부
```

Command — 업데이트와 이동을 한 번에

Core

핵심 정리

- update → State 변경 (Reducer 규칙대로 합쳐짐)
- goto → 다음 노드 지정 (조건부 엣지 대신)
- graph: Command.PARENT

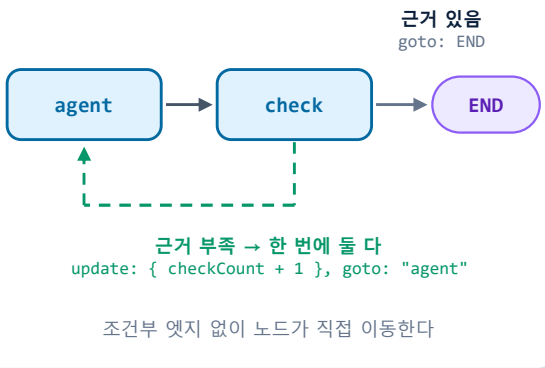
  → 바깥 그래프의 노드로 이동

```
import { Command } from "@langchain/langgraph";

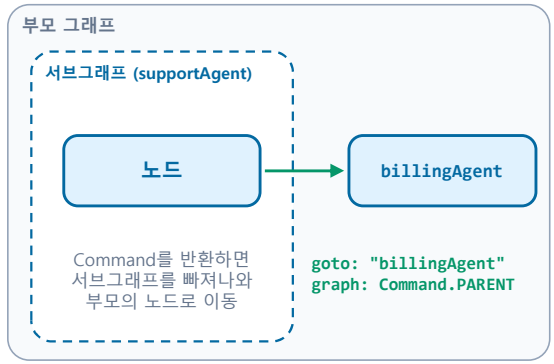
// ① 검증 노드가 다음 노드 지정 (goto)
const check: GraphNode<typeof SupportState> = async (state) =>
  isGrounded(state)
    ? new Command({ goto: END })
    : new Command({ update: { checkCount: state.checkCount + 1 },
                    goto: "agent" });

// ② 다른 에이전트에게 넘길 때
new Command({ goto: "billingAgent", graph: Command.PARENT });
```

① goto — 노드가 다음 노드를 고른다



② graph: PARENT — 바깥 그래프로



StateGraph 구축 5단계 패턴

구축 순서

- 1단계: State 스키마 정의 (StateSchema)
- 2단계: new StateGraph(State) 인스턴스 생성
- 3단계: addNode()로 노드 함수 등록
- 4단계: addEdge() / addConditionalEdges()로 연결
- 5단계: compile()로 실행 가능한 그래프 생성

compile 이후

- await invoke(): 실행 후 최종 상태 반환
- stream(): 노드별 중간 결과 스트리밍
- batch(): 여러 입력을 일괄 처리
- JS는 모든 실행이 비동기 (Promise · for await)

run.ts — compile 이후 실행

```
// graph: 앞의 'Edge — 흐름 제어' edge.ts에서
compile()한 그래프

// invoke: 실행 후 최종 상태 반환
const result = await graph.invoke({ n: 0 });
console.log(result);  // { n: 3 }

// stream: 노드별 중간 결과
const stream = await graph.stream({ n: 0 });
for await (const chunk of stream) {
  console.log(chunk); // { addOne: { n: 1 } }
}                     // { addOne: { n: 2 } } ...

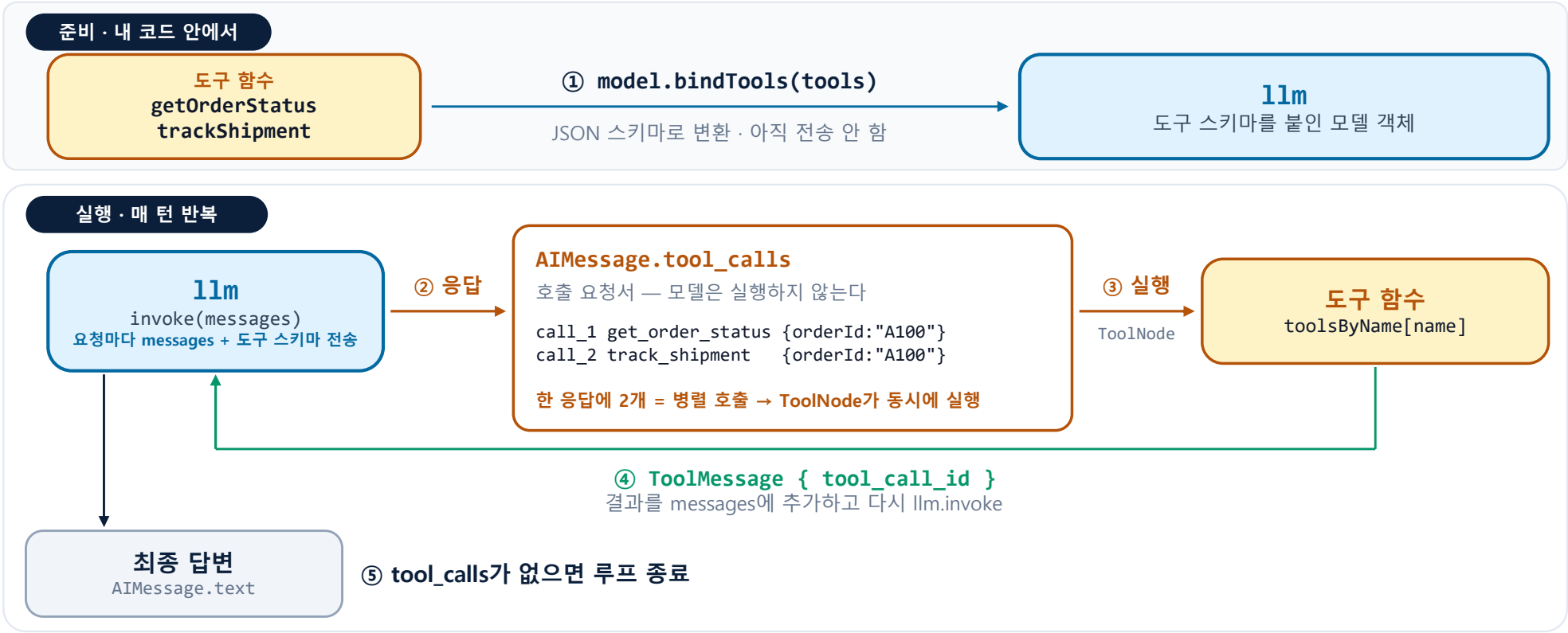
// batch: 여러 입력을 한 번에
const out = await graph.batch([{ n: 0 }, { n: 2 }]);
console.log(out);     // [{ n: 3 }, { n: 3 }]
```

Simple Graph 작성 및 실행하기

Module 03 | 가장 작은 에이전트 그래프를 조립하고 실행하기

bindTools와 tool_calls 구조

bindTools = 도구 스키마를 모델 객체에 붙여 둔다 (API 호출 없음) **tool_calls** = 모델이 보낸 호출 요청서



createAgent (안의 ToolNode)가 ②~④를 tool_calls가 없을 때까지 대신 반복한다

가장 작은 에이전트 그래프 — 조립하고 실행하기

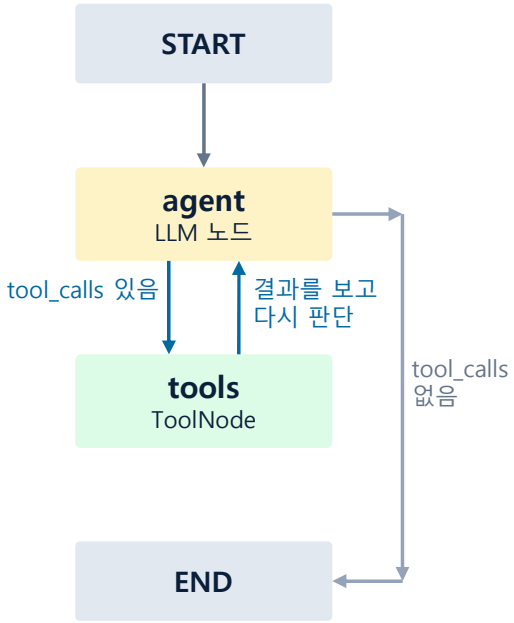
Code

```
// labs/02_agent_graph/solution.ts
const State = new StateSchema({ messages: MessagesValue });

const model = await initChatModel(MODEL);
const modelWithTools = model.bindTools(tools);   // 도구 붙이기

const agent: GraphNode<typeof State> = async (state) => {
  const response = await modelWithTools.invoke(state.messages);
  return { messages: [response] };
};

const graph = new StateGraph(State)
  .addNode("agent", agent)                          // ① LLM 노드
  .addNode("tools", new ToolNode(tools))            // ② 도구 노드
  .addEdge(START, "agent")                          // ③ 시작
  .addConditionalEdges("agent", toolsCondition,     // ④ 분기
    ["tools", END])
  .addEdge("tools", "agent")                        // ⑤ 루프
  .compile();
// human → ai(tool_calls) → tool → ai(답변)
```

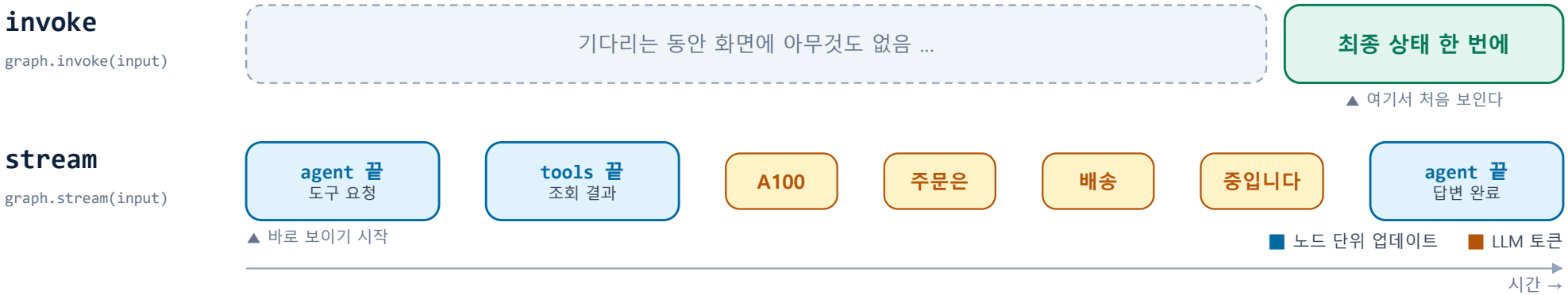


State · 도구(get_order_status · track_shipment) · agent 노드 · 그래프를 실습 파일 하나에서 직접 만든다 (agent 노드 안에서 bindTools)

Streaming — invoke vs stream

Core

왜 스트리밍? 응답이 늦을수록 체감 품질이 떨어진다 → '진행 중'을 보여 주면 기다림이 짧게 느껴진다



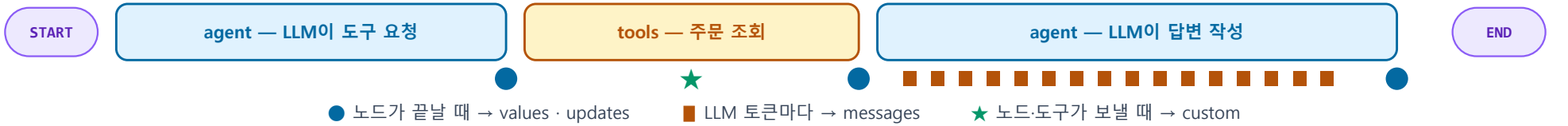
한눈에 비교

	invoke	stream
결과 시점	모두 끝난 뒤 한 번	노드 · 토큰마다 즉시
받는 값	최종 상태	단계별 업데이트 · 토큰
어울리는 곳	배치 · 테스트	챗 UI · 진행률 표시

```
// stream은 조각을 하나씩 돌려준다 — for await로 받는다
for await (const chunk of await graph.stream(input, { streamMode: "updates" })) { … }
```

streamMode — 어떤 조각을 받을까

같은 실행(agent → tools → agent)을 모드마다 다른 조각으로 받는다



values

노드가 끝날 때마다 State 전체
어울리는 곳: 상태 전체 화면 · 디버깅

```
{ messages: [Human, AI] }
{ messages: [Human, AI, Tool] }
{ messages: [Human, AI, Tool, AI] }
```

updates

노드가 바꾼 부분만 (노드 이름별)
어울리는 곳: 어느 단계까지 왔는지 진행 표시

```
{ agent: { messages: [AI(tool_calls)] } }
{ tools: { messages: [ToolMessage] } }
{ agent: { messages: [AI("A100 주문은 …")] } }
```

messages

LLM 토큰 하나씩 + 메타데이터
어울리는 곳: 채팅 타자 효과 · langgraph_node로 agent 토큰만

```
[AIMessageChunk("A100"),   { langgraph_node: "agent" }]
[AIMessageChunk("주문은"), { langgraph_node: "agent" }]
…
```

custom

노드·도구가 직접 보낸 값
어울리는 곳: 긴 작업의 진행 메시지

```
// 노드·도구 안에서
config.writer?.({ progress: "주문 조회 중 50%" });
→ { progress: "주문 조회 중 50%" }
```

여러 모드를 함께: `streamMode: ["updates", "messages"]` → 조각이 [모드, 값] 쌍으로 온다

LangSmith란?

LangSmith 개요

- LLM 애플리케이션의 개발·테스트·모니터링을 위한 통합 DevOps 플랫폼
- 무료 플랜(개인)부터 Enterprise까지 제공

부가 기능

- **Studio** — 그래프 보기 · 실행 · 디버깅 (npm run dev)
- **Fleet** (구 Agent Builder) — 노코드로 에이전트 만들기
- **Annotation Queues** — 사람이 결과 검토 · 라벨링

핵심 기능 4가지

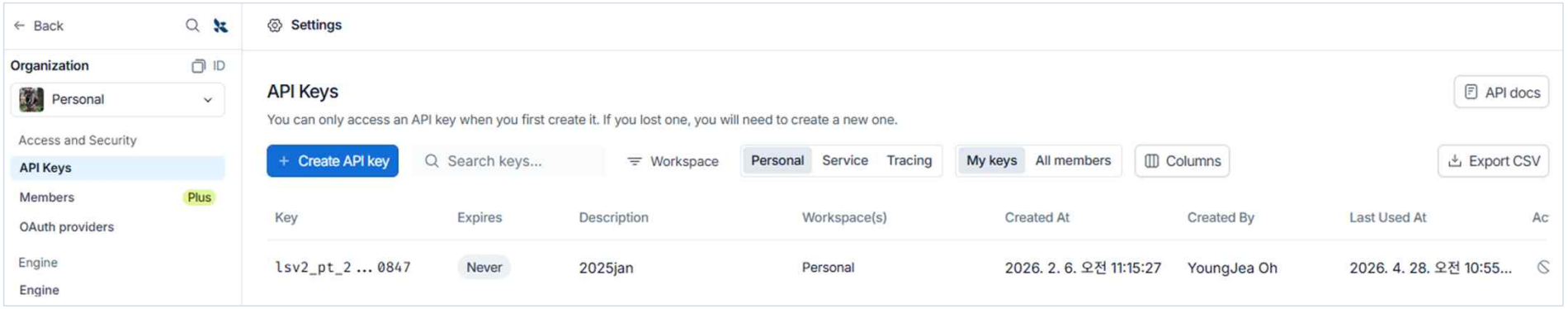
1) Observability (관찰)
트레이싱으로 모든 단계 기록 · 대시보드 · 알림

2) Evaluation (평가)
데이터셋 · 평가자로 품질 측정, 실험 비교

3) Prompt engineering (프롬프트)
Playground에서 시험 · Context Hub로 버전 관리

4) Deployment (배포)
LangSmith Deployment (이 과정은 Express + Render)

API Key 발급 smith.langchain.com → Settings → API Keys → + Create API key (.env의 LANGSMITH_API_KEY에 넣는다)



그래프 보기 — Mermaid vs LangSmith Studio

Mermaid — 구조만 빠르게 보기

계정 · 서버 없이 코드 몇 줄 (정적인 그림)

```
const g = await graph.getGraphAsync();
console.log(g.drawMermaid()); // 텍스트 출력
```

① 출력된 Mermaid 텍스트를 복사
② https://mermaid.live 에 붙여넣기 → 그림 완성
PNG 파일이 필요하면 drawMermaidPng()
✓ 노드 · 연결 구조 확인, 문서 · 발표 자료용

LangSmith Studio — 실행하며 디버깅

LangSmith 무료 계정 + 로컬 Agent Server

```
# .env
LANGSMITH_API_KEY=lsv2_...
npm run dev        # = langgraphjs dev
```

① 터미널에 나온 Studio 주소로 접속
https://smith.langchain.com/studio/?baseUrl=http://127.0.0.1:2024
② 로그인 → agent_graph · self_check · support 선택
③ 다 쓰면 터미널에서 Ctrl+C로 종료

할 수 있는 일	Mermaid (drawMermaid)	LangSmith Studio
노드 · 연결 구조	✓	✓ (같은 그림)
그래프 직접 실행 (질문 입력)	✗	✓
단계마다 State 값 (messages 등)	✗	✓
도구 호출 인자와 결과	✗	✓
thread(대화) 이력, 이어서 대화하기	✗	✓
중간에 멈추기 · 승인(interrupt) · 과거 단계로 되돌리기	✗	✓
실행 추적(trace): 모델 호출 시간 · 토큰	✗	✓

Studio에 그래프 올리기 — 필요한 구조

① 그래프를 export

```
import { createAgent, humanInTheLoopMiddleware } from "langchain";

// Studio가 찾는 것: export된 그래프 객체
export const graph = createAgent({
  model: MODEL,
  tools: [askOrderAgent, askPolicyAgent, refundOrder],
  systemPrompt: "너는 스마트몰 상담 총괄이야. …",
  middleware: [
    humanInTheLoopMiddleware({
      interruptOn: {
        refund_order: { allowedDecisions: ["approve", "reject"] },
      },
    }),
  ],
});
```

② langgraph.json — 이름 : 파일 경로 : export 이름

```
{
  "dependencies": ["."],
  "graphs": {
    "00_first_agent": "./labs/00_first_agent/studio.ts:agent",
    "02_agent_graph": "./labs/02_agent_graph/studio.ts:graph",
    "04_self_check": "./labs/04_self_check/studio.ts:graph",
  },
  "env": ".env"
}
```

③ Studio에서 graph 실행

export const graph = …
createAgent 결과 또는 .compile()한 그래프

→

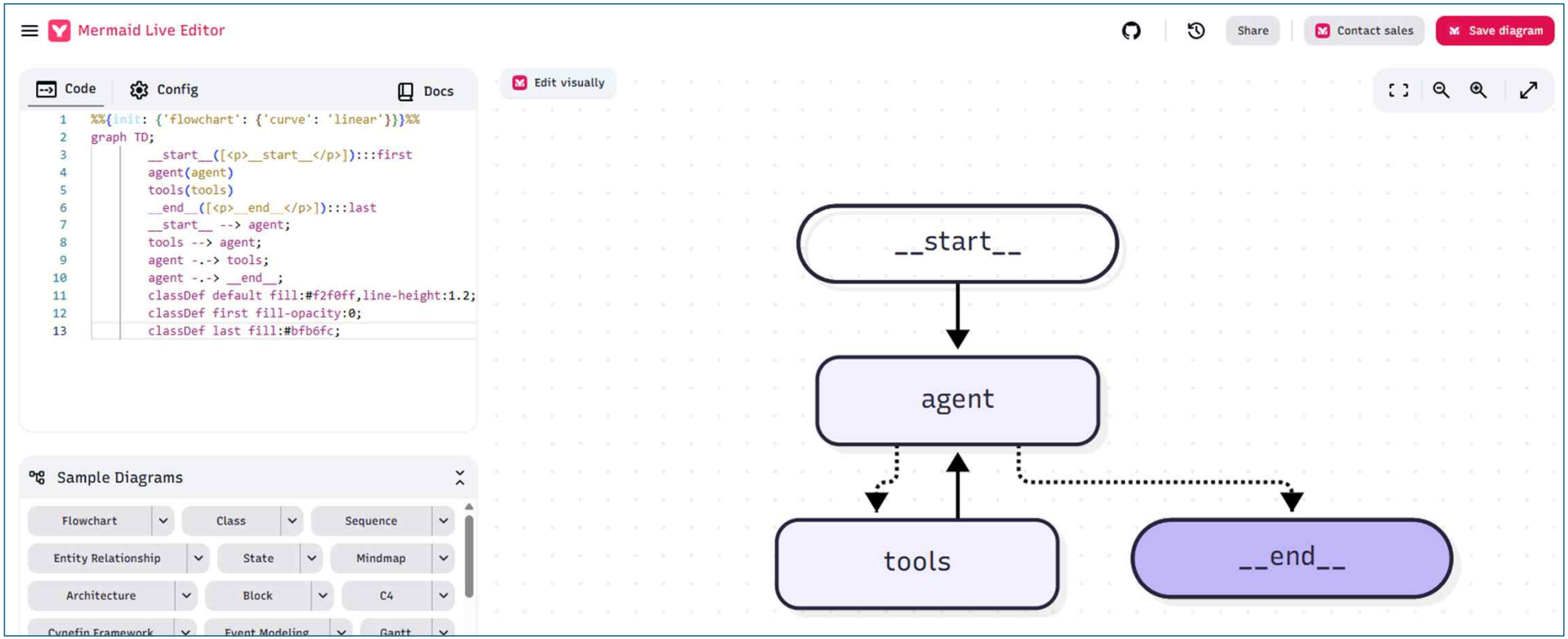
langgraph.json에 등록

→

npm run dev
Studio 그래프 목록에 support 표시

✓ **export 이름 = 경로 뒤 :이름** — 다르면 목록에 안 뜬다
✓ **실행 코드는 넣지 않기** invoke · console.log 없이 그래프만 export (실행은 Studio가 한다)
✓ **checkpointer는 생략** Agent Server가 저장소를 붙여 준다 → thread 이력 · interrupt 승인이 Studio에서 바로 동작
✓ **API 키는 .env** "env": ".env"로 OPENAI_API_KEY · LANGSMITH_API_KEY를 서버가 읽는다

Mermaid Live Editor (https://mermaid.live)



실습 : labs/02_agent_graph

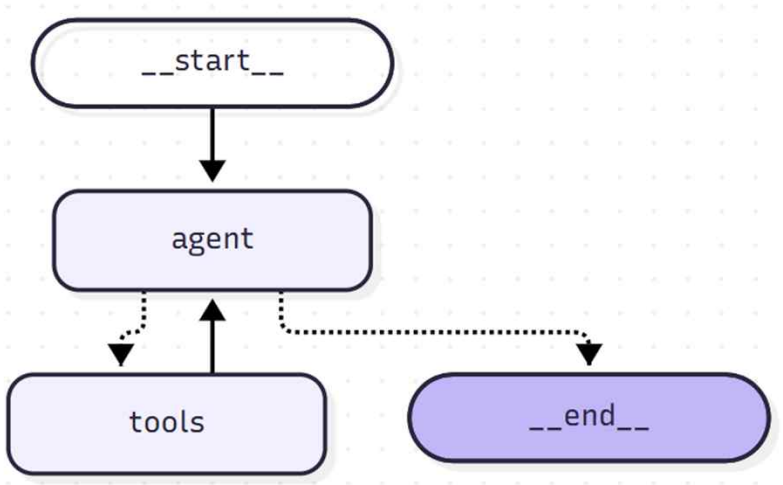
Hands-on

실습 목표

- 상담 에이전트의 agent ⇄ tools 루프를 StateGraph로 조립
- invoke vs updates 스트림 — 첫 출력까지 걸린 시간 비교
- messages 스트림으로 토큰 출력 · 첫 토큰 시간
- Studio · drawMermaid로 구조 확인

진행 순서

1. TODO 1: 그래프 조립 (toolsCondition)
2. TODO 2: updates 스트림 — 노드 · 경과 시간
3. TODO 3: messages 스트림 — 토큰 · 첫 토큰 시간
4. npx tsx labs/02_agent_graph/exercise.ts
5. npm run dev → Studio에서 실행

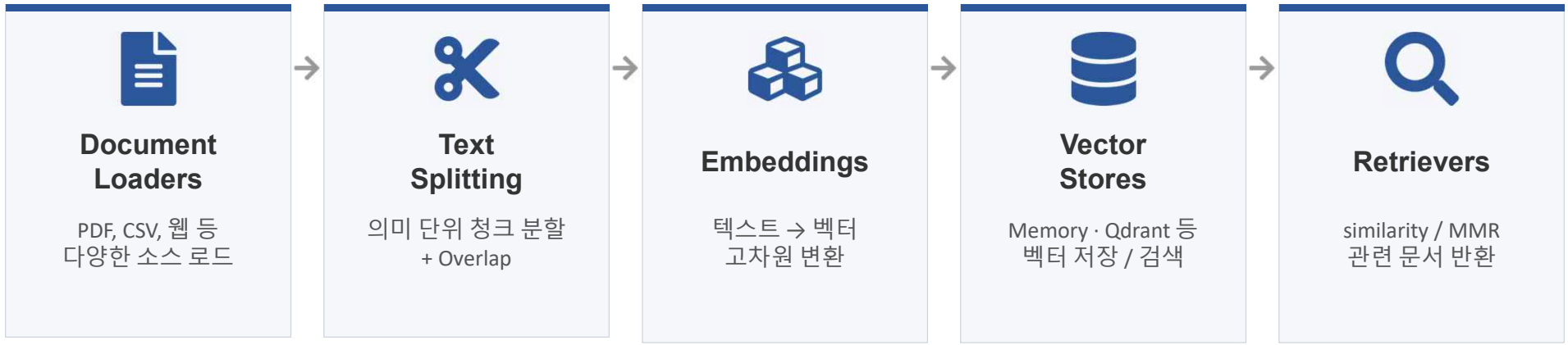


Retrieval Agent 작성하기

Module 04 | 검색을 에이전트의 도구로 통합하기

Semantic Search (시맨틱 검색 엔진 구축)

키워드 매칭이 아닌 의미 기반 검색 — 임베딩으로 텍스트를 벡터로 변환하고, 유사도 기반으로 관련 문서를 찾습니다.





품질을 좌우하는 3요소

① 분할(Chunking) — 의미 단위로 잘라야 검색이 잘 됨

② Retriever 전략 — similarity / MMR, 필터, 메타데이터

③ Prompt — 검색 결과를 "어떻게 쓰게 할지"

참고: LangChain 공식 문서 — Semantic Search

Document Loaders

- RAG 시스템에서 다양한 소스의 문서를 불러와 처리 가능한 형식으로 변환하는 기능을 수행. 이를 통해 외부 데이터 소스에서 정보를 효율적으로 수집하고, LLM(대규모 언어 모델)이 활용할 수 있도록 지원

LangChain JS가 제공하는 Document Loader (docs.langchain.com 통합 목록 · npm 패키지 기준)

분류	대표 로더	불러오는 대상	패키지
텍스트 · JSON	`TextLoader, JSONLoader, JSONLinesLoader`	.txt · .md · .json · .jsonl	`@langchain/classic`
폴더 · 여러 파일	`DirectoryLoader, MultiFileLoader`	확장자별 로더로 한 번에 읽기	`@langchain/classic`
문서 파일	`PDFLoader, CSVLoader, DocxLoader, PPTXLoader, EPubLoader`	PDF · CSV · Word · PowerPoint · EPUB	`@langchain/community`
웹 페이지	`CheerioWebBaseLoader, PlaywrightWebBaseLoader, RecursiveUrlLoader, SitemapLoader`	정적 HTML · JS 렌더링 · 하위 링크 · 사이트맵	`@langchain/community`
협업 도구	`NotionAPILoader, ConfluencePagesLoader, GithubRepoLoader, JiraProjectLoader`	Notion · Confluence · GitHub · Jira	`@langchain/community`
클라우드 저장소	`S3Loader, AzureBlobStorageContainerLoader, GoogleCloudStorageLoader`	AWS S3 · Azure Blob · Google Cloud Storage	`@langchain/community`
오디오 · 영상	`YoutubeLoader, OpenAIWhisperAudio`	YouTube 자막 · 음성을 텍스트로	`@langchain/community`

문서 분할 (Text Splitter)

왜 나누나

- 모델 입력 길이 한도를 넘지 않게 한다
- 작은 조각일수록 질문과 꼭 맞는 부분을 찾기 쉽다
- 조각마다 출처(metadata)가 남아 답변에 출처를 밝힐 수 있다
- 너무 작으면 문맥이 끊기고, 너무 크면 엉뚱한 내용이 섞인다

분할 전략

LangChain.js 내장 분할기: @langchain/textsplitters

전략	기준	특징	LangChain 클래스
길이 기준	글자 · 토큰 수	구현이 간단, 조각 크기가 일정	`CharacterTextSplitter` · `TokenTextSplitter`
텍스트 구조 기준	문단 → 문장 → 단어	자연스러운 흐름 유지 ← 이번 실습	`RecursiveCharacterTextSplitter`
문서 구조 기준	Markdown · HTML · 코드	원래 논리 구조(제목 · 섹션) 보존	`MarkdownTextSplitter` · `LatexTextSplitter` · `RecursiveCharacterTextSplitter.fromLanguage("html")`

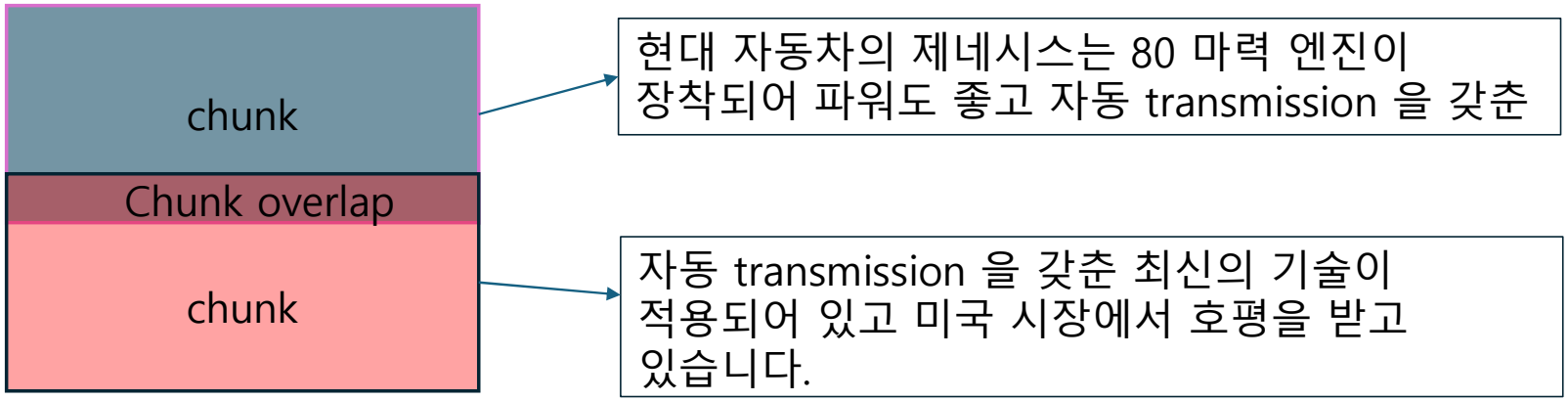
실습 03 — RecursiveCharacterTextSplitter

문단 → 문장 → 단어 순으로 잘라 300자 이하 조각을 만든다 (겹침 50자) · 조각마다 metadata.source가 그대로 남는다

Chunk Overlap

- 텍스트 분할기는 일부 청크가 겹치는 청크 분할 포함

Ex) 현대 자동차의 제네시스는 80 마력 엔진이 장착되어 파워도 좋고 자동 transmission 을 갖춘 최신의 기술이 적용되어 있고 미국 시장에서 호평을 받고 있습니다.



질문 : 제네시스의 자동차 사양에 대해 설명해줘

실습 03: RecursiveCharacterTextSplitter({ chunkSize: 300, chunkOverlap: 50 }) — 앞 조각의 끝 50자를 다음 조각 앞에 겹친다

Vector Stores — 실습은 Memory, 운영은 외부 DB

벡터 스토어

- 문서 조각을 임베딩(벡터)으로 저장하고, 유사도로 비슷한 조각을 찾는다
- 임베딩 모델과 함께 초기화한다

종류

- **인메모리형** — 설치 · 서버 없이 메모리에서 (MemoryVectorStore, 이번 실습)
- **독립 서버형** — 자체 서버에 영구 저장 (Qdrant · Redis · pgvector)
- **호스팅형** — 클라우드가 운영 (Pinecone · Qdrant Cloud · MongoDB Atlas)

	MemoryVectorStore (실습)	Qdrant 등 외부 DB (운영)
저장 방식	메모리에만 저장 (종료하면 사라짐)	디스크(컬렉션)에 영구 저장
설치 · 실행	@langchain/classic, 서버 없음	@langchain/qdrant + Qdrant 서버
확장성	테스트 · 소규모 문서	대규모 데이터 · 필터 검색
데이터 공유	실행마다 다시 임베딩	여러 서버가 같은 데이터 공유

운영으로 옮길 때 바꾸는 곳은 벡터 스토어를 만드는 한 줄

- MemoryVectorStore.fromDocuments(...) → QdrantVectorStore.fromDocuments(..., { url, collectionName })
- asRetriever() · similaritySearchWithScore() · search_policy 도구 코드는 그대로 — 같은 VectorStore 인터페이스
- 이번 과정은 설치 없는 MemoryVectorStore로 진행

참고: Chroma는 JS에서 @langchain/community(커뮤니티 패키지)로 제공된다

검색기 (Retrievers)

Retriever = 질문을 넣으면 Document[]를 돌려주는 검색 규격 (Runnable)

VectorStore = 벡터를 저장 · 삭제 · 검색하는 저장소

배경 VectorStore가 먼저 생겼고, 2023년 **비벡터 검색(웹·키워드·API)도 끼우려고 Retriever 추가** → `asRetriever()`는 그 어댑터

언제 무엇을 쓰나 — 결정 요소

결정 요소	VectorStore 직접	Retriever
문서 넣기 · 삭제 (인덱싱)	✓ `fromDocuments` · `addDocuments`	✗ 읽기 전용
에이전트 도구에서 검색	가능	✓ **기본** · `retriever.invoke(query)`
유사도 점수가 필요할 때	✓ `similaritySearchWithScore`	✗ 문서만 돌려준다
검색 전략 추가 (mmr · 재순위화 등)	메서드를 바꿔 부르거나 직접 구현	✓ 감싸기만 하면 됨 — 도구 코드 그대로
벡터가 아닌 검색 (웹 · 키워드 · API)	✗ 불가	✓ 같은 invoke(query)로
LangSmith 추적 · 체인/도구 연결	✗ Runnable 아님	✓ Runnable이라 바로 연결
실습 예	03 · 04 인덱싱, 04 self_check (점수 판단)	03 RAG 에이전트의 검색 도구

기본은 Retriever, 점수를 봐야 하면 VectorStore — 찾아오는 문서는 같고, 점수가 붙느냐만 다르다

withStructuredOutput vs responseFormat

Core

	withStructuredOutput	responseFormat
선택 기준	모델을 직접 부를 때	createAgent에 맡길 때
붙이는 곳	`model.withStructuredOutput(Schema)`	`createAgent({ …, responseFormat })`
도구 루프	없음 — 호출 1회	있음 — 도구를 쓴 뒤 답변을 객체로
OpenAI로 가는 요청	둘 다 같다 — zod → JSON Schema → `text.format (json_schema · strict)`	

```
model.withStructuredOutput(PolicyAnswer);


const agent = createAgent(
{ model,
  tools: [searchPolicy],
  systemPrompt,
  responseFormat: PolicyAnswer
});
```

OpenAI Responses API 요청 본문 (일부)

```
"text": { "format": {
  "type": "json_schema", "name": "extract", "strict": true,
  "schema": {
    "type": "object",
    "properties": {
      "answer":     { "type": "string", "description": "고객에게 보낼 답변" },
      "sources":    { "type": "array", "items": { "type": "string" }, … },
      "needsHuman": { "type": "boolean", "description": "…" }
    },
    "required": ["answer", "sources", "needsHuman"],
    "additionalProperties": false
} } }
```

실습 : labs/03_RAG_agent — 검색 도구 → RAG 에이전트

Hands-on

실습 목표

- 정책 문서 5개 로드 (배송 · 환불 · 회원 · 결제 · 제품)
- TextSplitter로 300자 · 겹침 50자 분할
- MemoryVectorStore + asRetriever({ k: 3 })
- retriever.invoke로 search_policy 도구 만들기
- createAgent + responseFormat → RAG 에이전트

핵심 — 검색기 → 도구 → RAG 에이전트

```
const vectorStore = await MemoryVectorStore
  .fromDocuments(chunks, embeddings); // ← 운영: 이 줄 교체
const retriever = vectorStore.asRetriever({ k: 3 });

// 검색 도구를 가진 에이전트 = RAG 에이전트
const agent = createAgent({
  model: MODEL, tools: [searchPolicy],
  systemPrompt: "검색 결과만 근거로 답하고 출처 파일명 표시…" });
```

진행 순서

1. TODO 1: 문서 → Document (출처 메타데이터)
2. TODO 2: 분할 (300자 · 겹침 50자)
3. TODO 3: MemoryVectorStore · asRetriever
4. TODO 4: retriever.invoke → search_policy
5. TODO 5: createAgent(도구 · systemPrompt) 질문
6. TODO 6: responseFormat으로 구조화 응답
7. npx tsx labs/03_RAG_agent/exercise.ts

확인: [tool] 검색 결과 다음 [ai] 답변에 출처가 붙는가, structuredResponse.sources에 파일명이 담기는가

답변을 검증하는 Self-RAG 구축하기

Module 05 | 검증 노드로 답변의 근거를 확인하고 다시 시도하기

Self-RAG — 에이전트가 자기 답변을 검증한다

Learn

왜 검증이 필요한가

- 검색 결과가 질문과 무관할 수 있다
- 모델이 정책 문서에 없는 내용을 지어낼 수 있다 (환각)
- 고객에게 틀린 환불 기간을 안내하면 실제 손해가 된다
- → 답변을 내보내기 전에 검증 노드를 둔다

① 검색 필요성

Retrieve?

에이전트가 판단
(Module 04)

② 문서 관련성

Relevant?

가져온 문서가 질문과
관련 있는가

③ 근거성

Grounded?

답변이 도구 결과에
근거하는가 (환각 검사)

④ 유용성

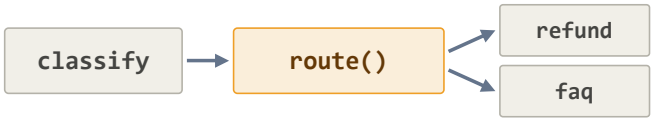
Useful?

답변이 질문을
해결하는가

조건 분기 두 가지 방법 — 조건부 엣지 vs Command

두 방법 모두 몇 갈래든 분기할 수 있다 — **기본은 조건부 엣지**, 바꾸면서 이동해야 하면 **Command**

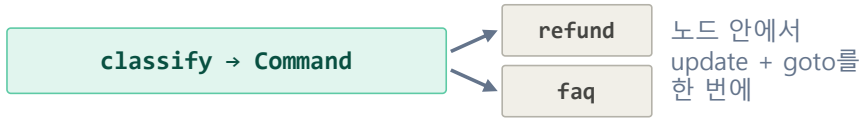
① 조건부 엣지 (Conditional Edge)




```
// 노드는 일만 한다 (길은 모른다)
graph.addNode("classify", classify)
  .addConditionalEdges("classify", route,
    ["refund", "faq"]);            // 갈 수 있는 곳

// 노드 밖 라우터: State를 읽고 이름만 돌려준다
const route = (state) =>
  state.intent === "환불" ? "refund" : "faq";
```

② Command (goto)

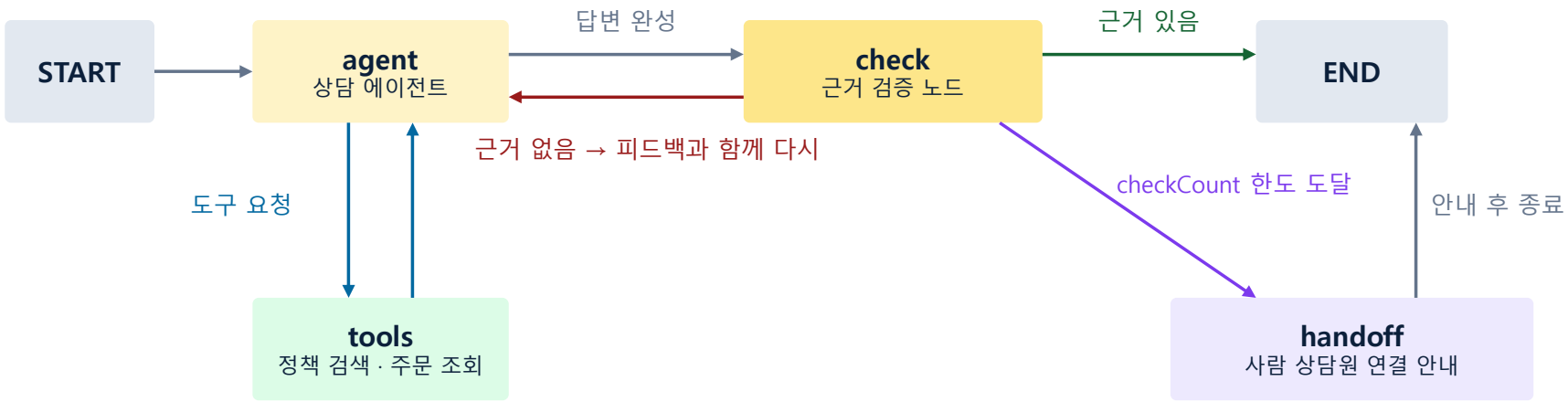



```
const classify = async (state) => {
  const intent = await detectIntent(state);
  return new Command({ update: { intent },     // State 변경
    goto: intent === "환불" ? "refund" : "faq" });  // + 갈 곳
};

graph.addNode("classify", classify,
  { ends: ["refund", "faq"] });      // 그림용 후보
```

	① 조건부 엣지	② Command
길을 정하는 곳	노드 밖 라우터 함수	노드 안 — `return new Command({ goto })`
State 변경	노드가 먼저 바꾸고, 라우터는 읽기만	`update` + `goto`를 한 번에
장점	구조가 그래프 정의에 다 보임 · 노드 재사용 쉬움	판단과 이동이 한곳에 · 할 수 있는 일이 많음
이럴 때 쓴다	기본 선택 — 길만 고를 때 예) 고칠 수 없는 노드(`ToolNode` 등) 뒤 · `START`에서 바로 분기	State를 바꾸며 이동 예) 부모 그래프로 이동(handoff) · 도구에서 State 변경 · `resume`

검증 노드가 붙은 에이전트 그래프 — 구조와 코드



check 노드 ① 판정 — Verdict + withStructuredOutput

```
const Verdict = z.object({
  grounded: z.boolean()
    .describe("도구 결과에만 근거하면 true"),
  reason: z.string()
    .describe("근거가 부족하면 무엇이 부족한지"),
});
const grader = model.withStructuredOutput(Verdict);
const verdict = await grader.invoke(prompt); // 근거 + 답변
```

check 노드 ② 분기 — Command로 세 갈래

```
if (verdict.grounded)
  return new Command({ goto: END });          // 근거 있음
if (state.checkCount + 1 >= MAX_CHECKS)
  return new Command({ goto: "handoff" });    // 한도 도달
return new Command({                          // 근거 없음
  update: { checkCount: state.checkCount + 1,
    messages: [new HumanMessage(
      `[검증 실패] ${verdict.reason}`)] },
  goto: "agent",
});
```

실습 : labs/04_self_check

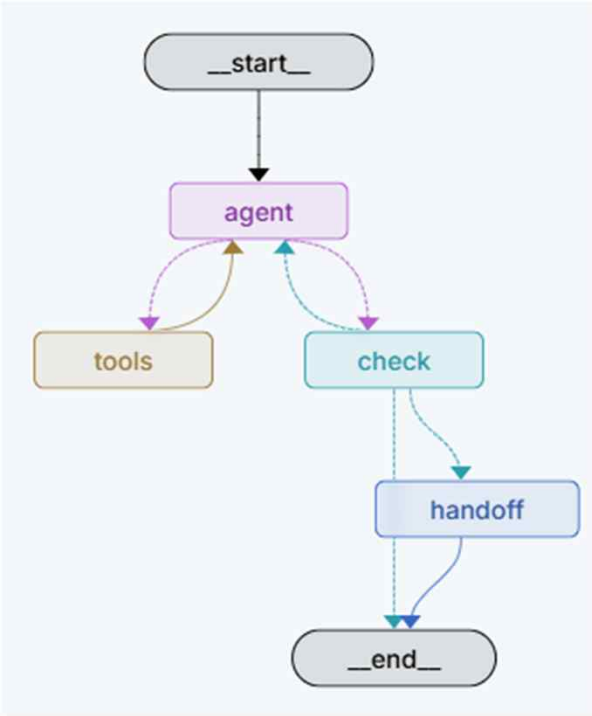
Hands-on

실습 목표

- Verdict 스키마로 근거성 판정 (구조화 출력)
- check 노드: 통과 · 재시도 · 사람 연결을 Command로 결정
- 실패하면 피드백 메시지와 함께 agent로 되돌리기
- checkCount 한도로 무한 루프 방지

진행 순서

1. TODO 1: Verdict 스키마
2. TODO 2: check 노드
3. TODO 3: agent 다음 라우팅 (tools | check)
4. 테스트 질문으로 check 경로(통과 · 재시도 · handoff) 확인
5. npx tsx labs/04_self_check/exercise.ts



테스트 질문: "VIP는 환불 기간이 1년이죠?" → 정책은 "7일 이내 · 모든 등급 같음" → 틀린 답이면 grounded: false로 재시도, 2번 실패하면 상담원 연결 안내

Checkpoint와 영속성(Persistence) 구현하기

Module 08 | 고객별 대화 기억 · 체크포인트 · Time Travel

Memory: “대화”를 상태로 다루기

Memory = 에이전트가 이전 상호작용을 기억하는 방법 — 기억하는 **범위**에 따라 단기 · 장기로 나눈다

	단기 메모리 (Short-term)	장기 메모리 (Long-term)
기억하는 것	한 대화의 흐름 — `messages` · 도구 결과 등 상태 전체	사용자 선호 · 프로필 · 학습 이력 (JSON)
범위	같은 `thread_id` 안에서만 (채팅방 1개)	여러 thread에 걸쳐 (사용자 단위)
비유	카카오톡 채팅방의 대화 기록	카카오톡 프로필 정보
구현 도구	Checkpointer: `MemorySaver` , `SqliteSaver` · `PostgresSaver`	Store: `InMemoryStore`, `PostgresStore`
저장 · 불러오기	스텝마다 자동 저장 · `thread_id`로 자동 복원	`store.put()`으로 저장 · 필요할 때 `store.search()`
주의	대화가 길어지면 `messages` 정리 (요약 · 자르기)	개인정보 · 보안 정책이 핵심

Checkpointer

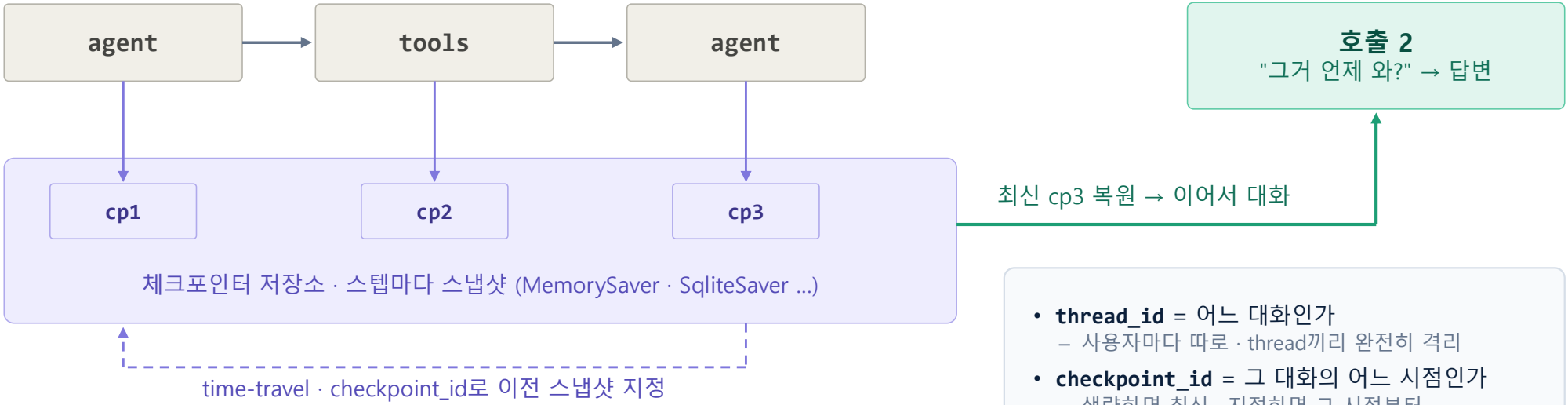
Checkpointer = 에이전트가 한 스텝 실행될 때마다 그 시점의 전체 상태를 스냅샷으로 저장하고 복원하는 컴포넌트

Checkpointer 없음

호출 1 · "A100 주문 어디?"
답변 후 상태 사라짐

호출 2 · "그거 언제 와?"
빈 상태에서 시작 → 이전 맥락 없음 · 오답

Checkpointer 있음 · thread_id = "user-42"



- **thread_id** = 어느 대화인가
  – 사용자마다 따로 · thread끼리 완전히 격리
- **checkpoint_id** = 그 대화의 어느 시점인가
  – 생략하면 최신 · 지정하면 그 시점부터

MemorySaver · SqliteSaver · PostgresSaver

단기 메모리(Checkpointer) 구현 방법 — 같은 thread_id 안에서만 기억하고 **저장 위치만 다르다** (장기 메모리는 Store)

	MemorySaver	SqliteSaver	PostgresSaver
저장 위치	프로세스 메모리 (RAM)	SQLite 파일 (`checkpoints.db`)	PostgreSQL 서버
영속	X — 프로세스 종료 시 소실	O — 서버 재시작 후에도 복원	O — 여러 서버가 함께 사용
속도	가장 빠름	빠름 (로컬 파일)	중간 (네트워크 거침)
용도	개발 · 테스트 · 실습	로컬 · 단일 서버	대규모 프로덕션 (여러 인스턴스)
패키지	`@langchain/langgraph`	`@langchain/langgraph-checkpoint-sqlite`	`@langchain/langgraph-checkpoint-postgres`
만들기	`new MemorySaver()`	`SqliteSaver.fromConnString("checkpoints.db")`	`PostgresSaver.fromConnString(url)` + `await setup()`

```
const graph = builder.compile({ checkpointer });   // 저장소만 바꾸면 나머지 코드는 그대로
```

실습 : labs/07_memory

Hands-on

실습 목표

- 체크포인터 없음 → 이전 대화를 기억 못 함
- MemorySaver + 같은 thread_id → 대화가 이어짐
- context로 고객 ID 전달 → 스토어에서 꺼내기
- InMemoryStore에 저장 → 고객 선호도 기억
- getStateHistory → 단계마다 남은 체크포인트 보기
- replay · fork → 과거 시점부터 다시 실행

진행 순서

1. TODO 1: 노드에서 스토어 값 읽기
2. TODO 2: checkpointer · store로 compile
3. TODO 3: 같은 thread_id로 대화 이어 가기
4. TODO 4: store.put으로 연락 방법 저장 후 확인
5. TODO 5: getStateHistory로 체크포인트 모으기
6. TODO 6: updateState로 fork → 그 갈래에서 실행
7. npx tsx labs/07_memory/exercise.ts

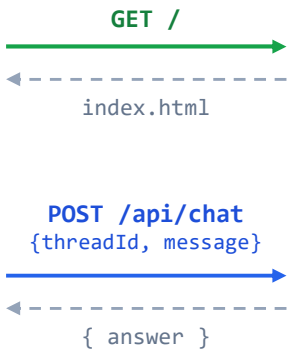
실습 : labs/07_memory/web — 터미널 챗봇을 웹 채팅으로

Hands-on



web/public/index.html
바닐라 JS — fetch로 JSON API 호출

「새 대화」 버튼
`crypto.randomUUID()`로
threadId만 새로 만든다 — 서버 호출 없음
대화 이력은 끊기고 store는 남는다



web/server.ts — Express 서버

GET /
express.static(public)
→ 채팅 화면 파일을 그대로 보낸다

POST /api/chat
① **입력 검사**
비었거나 500자 초과 → 400
② **graph.invoke**
새 메시지 1개만 전달
`thread_id = threadId`
`context = { C002 }`
③ **마지막 메시지 → { answer }**

StateGraph (solution.ts 복사)

START → agent → END

agent 노드가 하는 일
1. store에서 C002의 연락 방법을 읽어 시스템 프롬프트에 붙인다
2. 이전 대화 + 새 메시지로 모델 호출

`compile({ checkpointer, store })`
MODEL openai:gpt-6-luna

MemorySaver
checkpointer — thread_id별
대화 이력을 자동으로 저장 · 불러오기
서버를 끄면 사라진다

InMemoryStore
store — thread와 상관없는 장기 메모리
`preferences/C002 → 이메일`
서버가 시작될 때 put으로 저장

실행 `npm run web:07` → localhost:3000

핵심 브라우저는 threadId + 새 메시지 1개만 보낸다 — 이력은 checkpointer, 연락 방법은 store

Part A 정리

Summary

01

LangChain과 LangGraph 비교

createAgent로 첫 상담 에이전트

02

LangGraph 핵심 요소

State · 노드 설계, Context

03

Simple Graph

agent ⇄ tools 그래프 조립 · 스트리밍

04

Retrieval Agent

정책 문서 검색 도구

05

Self-RAG

답변 근거 검증 노드

06

도구(Tool) 활용

Context · Command · 오류 처리

07

Function Calling·Tool Use

병렬 호출 · 구조화 응답

08

Checkpoint·Persistence

고객별 기억 · 되감기

Multi-Agent의 개념과 장점 이해하기

Module 09 | 역할 분리와 오케스트레이션 구조

Multi-agent 패턴: “역할 분리 + 오케스트레이션”

왜 멀티에이전트인가?

- 한 에이전트에 도구/지식이 너무 많으면 품질이 흔들림

→ 컨텍스트 창을 과부하시키지 않고 전문 지식 제공

- 역할을 나누면 프롬프트/도구/컨텍스트가 깔끔

→ 여러 팀이 독립적으로 기능을 개발하고 유지 관리

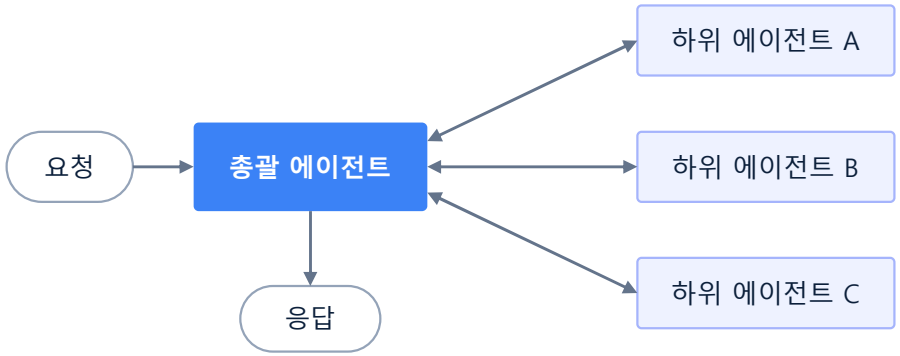
- 조율자는 "라우팅/핸드오프/검증"에 집중
- 대표 패턴 4가지는 다음 장 비교표에서 정리

설계 체크리스트

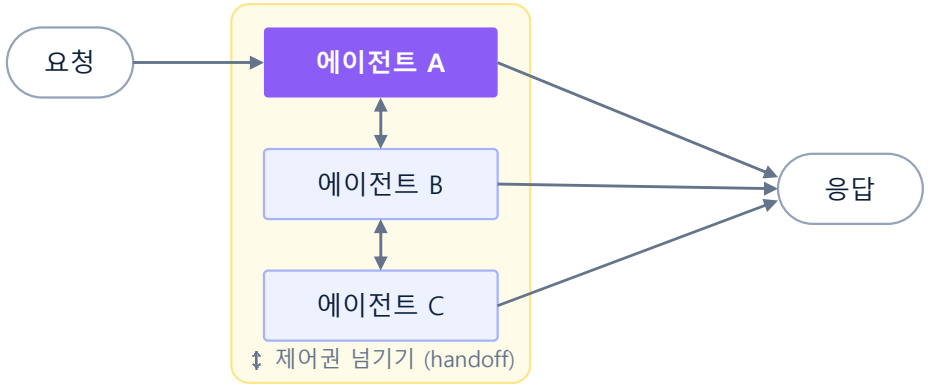
- 각 에이전트의 “책임 범위”를 한 문장으로 적는다
- 입력/출력 스키마를 정한다.
  (Structured Output 권장)
- 공유 상태는 최소화(필요한 정보만 전달)
- 실패 시 fallback 경로를 만든다

멀티 에이전트 패턴 흐름도

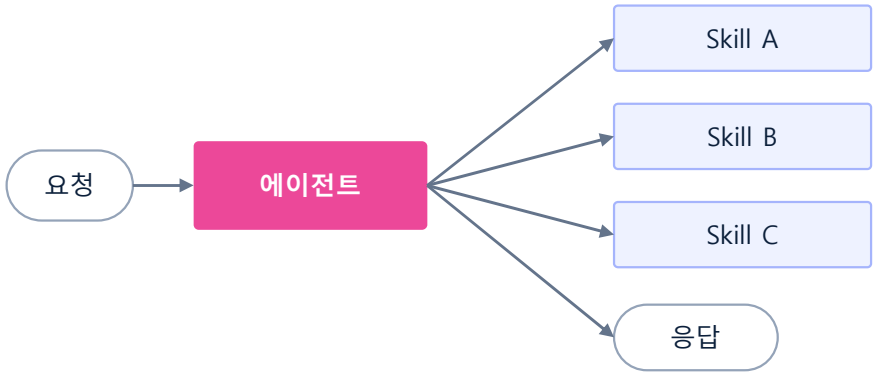
Subagents 총괄 에이전트가 하위 에이전트를 도구로 호출하고 결과를 돌려받는다 (⇄). 모든 흐름이 총괄을 거친다



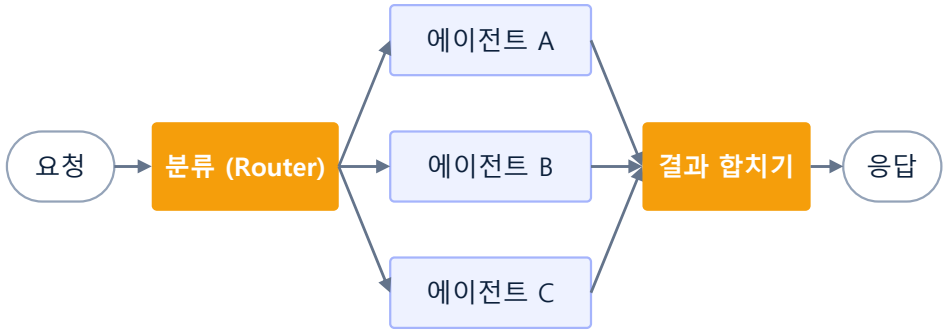
Handoffs 도구 호출로 제어권을 다른 에이전트에 넘긴다. 어느 에이전트든 사용자에게 직접 답할 수 있다



Skills 에이전트 하나가 주도하면서 필요할 때 전문 프롬프트 · 지식을 불러온다



Router 분류 단계가 입력을 하나 이상의 전문 에이전트로 보내고 결과를 합친다



출처: LangChain Docs — Multi-agent (Patterns)

멀티 에이전트 패턴 종합 비교

패턴	동작 방식	이럴 때 쓴다	이 과정
Subagents	총괄 에이전트가 하위 에이전트를 도구로 호출한다. 모든 라우팅은 총괄을 거친다.	일을 나눠 전문 에이전트에게 맡기고 총괄이 결과를 모아 답할 때. 하위 에이전트는 서로 독립적이라 따로 만들고 동시에 호출 가능. (예: 주문 · 결제 · 정책 담당)	실습 08a
Handoffs	도구 호출이 상태 값을 바꾸면 담당 에이전트(프롬프트)가 바뀐다.	대화 도중 담당자가 바뀌어야 할 때. 넘겨받은 에이전트가 고객과 직접 이어서 대화한다 (예: 주문 상담 중 결제 문의 → 결제 상담원에게 넘기기)	실습 08b
Skills	에이전트 하나가 주도하면서 필요할 때 전문 프롬프트 · 지식을 불러온다.	에이전트는 하나로 두고 필요한 지식만 그때그때 불러올 때. 평소엔 스킬 목록만 들고 있어 컨텍스트를 아끼고, 문서를 추가하면 기능이 는다 (예: 환불 질문에만 refund.md 불러오기)	실습 08c
Router	분류 단계가 입력을 하나 이상의 전문 에이전트로 보내고 결과를 합친다.	질문 종류가 뚜렷이 나뉠 때. 먼저 분류해 맞는 전문가에게 보내고, 여러 곳에 동시에 물어 결과를 하나로 합친다 (예: 주문 · 결제 · 정책 에이전트에 동시 질의)	실습 08d
Custom workflow	LangGraph로 정해진 흐름과 에이전트를 섞어 직접 설계한다. 다른 패턴을 노드로 넣을 수 있다.	업무 순서와 분기가 정해져 있을 때. 검증 · 승인 같은 고정 단계를 넣거나 다른 패턴을 노드로 조합해 흐름을 직접 짠다 (예: 답변 → 근거 검증 → 재시도 · 상담원 연결)	실습 02 · 04

출처: LangChain Docs — Multi-agent (Patterns · Choosing a pattern)를 바탕으로 정리

패턴은 섞어 쓸 수 있다
예: Subagents 총괄이 Router나 Custom workflow를 도구로 부르고, 하위 에이전트는 Skills로 필요한 컨텍스트를 불러온다.

Subagents: 스마트몰 상담 팀



전문 에이전트는 tool()로 감싸져 총괄의 '도구'가 된다 → 총괄은 도구 호출로 위임한다

tool()로 감싸기 = 전문 에이전트 실행을 도구 함수 안에 넣는다 → 총괄은 다른 도구처럼 호출만 한다

① 입력 = 요청 문장
총괄(LLM)이 맡길 일을 문장 하나로 써서 넘긴다

② 도구 안에서 실행
전문 에이전트가 자기 도구로 끝까지 처리한다

③ 최종 답만 돌려준다
중간 도구 호출은 총괄 대화에 남지 않는다 (컨텍스트 격리)

④ name · description
총괄이 어느 전문 에이전트에게 맡길지 고르는 기준

실습 미션 (labs/08a_subagents)
- 고객: "A100 배송이 너무 늦어요. 늦으면 환불돼요?"
- 총괄: 주문 · 배송 에이전트(배송 상태) → 제품 · 정책 에이전트(지연 환불 규정) 순서로 위임 → 답변 종합

실습 : labs/08a_subagents

Hands-on

실습 목표

- 주문 · 결제 · 제품 전문 에이전트 3개 구성
- 전문 에이전트를 tool()로 감싸 총괄의 도구로
- 총괄이 필요한 에이전트에만, 순서대로 위임하는지 확인

진행 순서

1. TODO 1: 전문 에이전트 3개 (도구 · 프롬프트 분리)
2. TODO 2: 에이전트 → 도구 래핑
3. TODO 3: 상담 총괄 생성
4. 미션 문장으로 위임 순서 확인
5. npx tsx labs/08a_subagents/exercise.ts
6. (선택) npm run dev → Studio에서 subagents

미션: "A100 배송이 너무 늦어요. 늦으면 환불돼요?"

도구에서 실행 정보 읽기 — ToolRuntime · Context

도구 함수는 두 가지를 받는다

누가 무엇을 넣는가: **앱** · **LangGraph** · **모델**

LangGraph (실행기)
도구를 부를 때 runtime을 만들어 넘김

모델 (LLM)
schema를 보고 인자를 채움

앱 (서버 코드)
`invoke(입력, { context })`

② **ToolRuntime** LangGraph가 넣어 주는 실행 정보

- **context** — 앱이 넘긴 고정 정보 (customerId)
- **state** — 지금의 State (messages)
- **toolCallId** — 이번 도구 호출의 id
- **store** — 장기 메모리 (있을 때)

① 인자 · 모델이 채움
`contact: "문자"`

고객 ID를 인자로 두면?
모델이 "C999"를 넣을 수도

③ 도구 함수에서 만남
`async ({ contact }, runtime) => …`
`runtime.context.customerId · runtime.state · runtime.toolCallId`

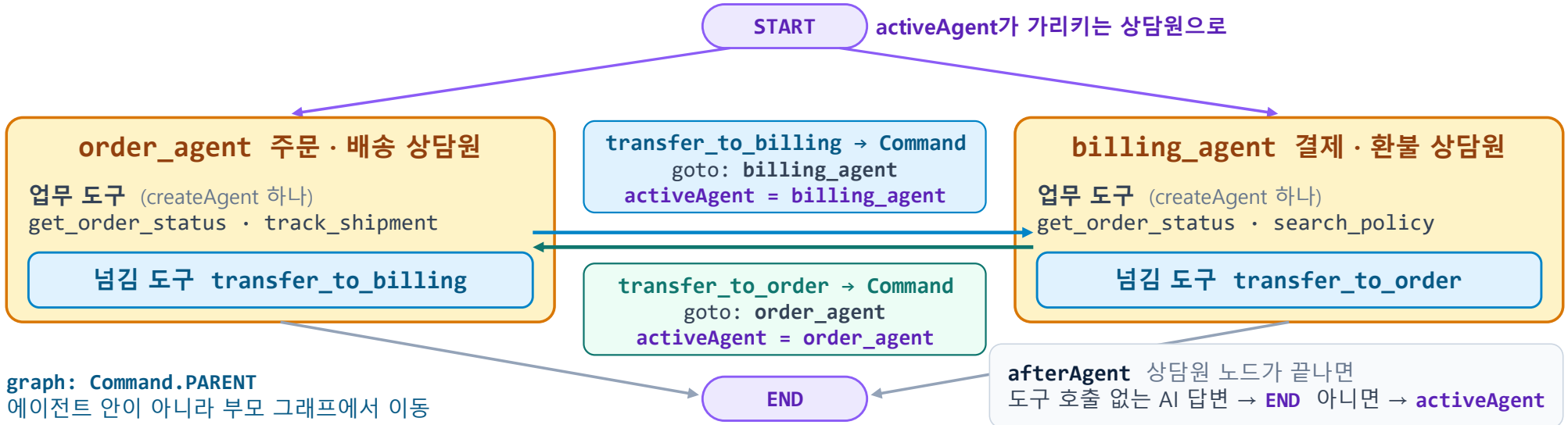
→ 고객 ID처럼 **바뀌면 안 되는 값**은 인자가 아니라 **Context**로
인자는 모델이 채우므로 다른 값이 들어갈 수 있다

Handoffs: 상담원끼리 대화 넘기기

HandoffState 두 상담원이 함께 쓰는 그래프 State

`messages`	대화 전체 — 넘김 메시지도 여기에 남는다
`activeAgent`	`"order_agent" | "billing_agent"` 지금 담당 상담원

MemorySaver (체크포인터)
thread별로 State를 저장 →
다음 질문에도 activeAgent가 남는다



`graph: Command.PARENT`
에이전트 안이 아니라 부모 그래프에서 이동

실습 08b 같은 thread에서 세 번 묻기

① A100 주문 어디쯤 왔어요?
`order_agent`가 바로 답한다
`activeAgent: order_agent`

② 지연되면 보상 · 환불 받나요?
`transfer_to_billing` → `billing_agent`
`activeAgent: billing_agent 로 바뀜`

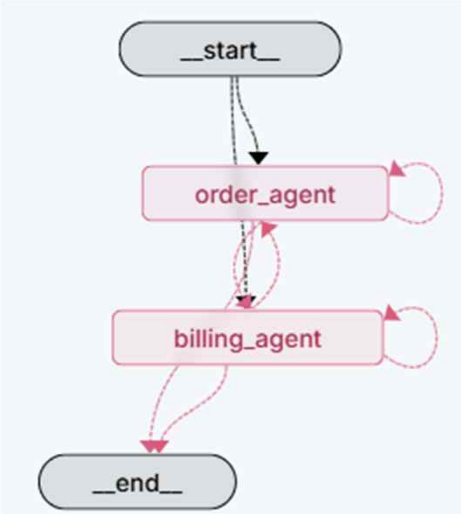
③ 환불하면 돈은 언제 들어와요?
`START`에서 바로 `billing_agent`
`activeAgent가 남아 있어서`

실습 : labs/08b_handoffs

Hands-on

실습 목표

- 주문 · 배송 / 결제 · 환불 상담원 2명이 대화를 서로 넘기기
- 넘김 도구가 Command로 부모 그래프의 다른 상담원 노드로 이동



진행 순서

1. TODO 1: 넘김 도구 2개 (Command.PARENT)
2. TODO 2: 상담원 2명 (업무 도구 + 넘김 도구)
3. TODO 3: 그래프 조립 (START → activeAgent)
4. 같은 thread에서 세 번 물어 넘김 확인
5. npx tsx labs/08b_handoffs/exercise.ts
6. (선택) npm run dev → Studio에서 08b_handoffs

미션: "A100 어디쯤 왔어요?" → "지연되면 환불돼요?" → "환불은 언제 들어와요?"

Middleware: 안전·비용·품질을 “훅”으로 제어

무엇을 할 수 있나?

- 에이전트 내부에서 일어나는 일을 더 세밀하게 제어
- 핵심 실행 루프의 각 단계(모델 호출, 도구 실행, 종료 처리)의 이전(before)과 이후(after) 시점에 개입할 수 있는 훅(hook)을 제공
- hook(훅) - 프로그래밍에서 특정 동작이 일어날 때 자동으로 실행되도록 연결된 코드 조각

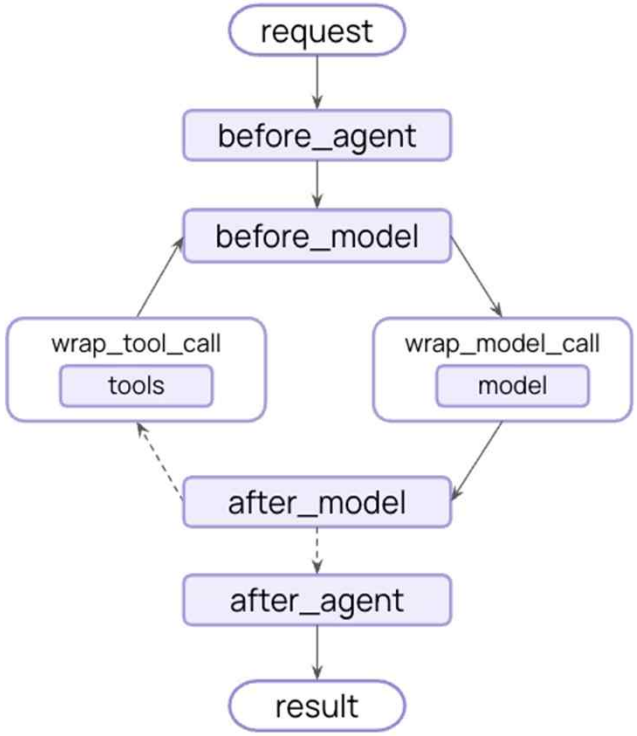
Before model

- 민감정보 차단(PII)
- 도구 호출 제한
- 모델 호출 횟수 제한

After model

- 응답 검열/포맷 검증
- 부적절 출력 제거

wrapToolCall / wrapModelCall → 도구/모델 실행 customize
(예: 08c skillMiddleware — 스킬 목록 덧붙이기)



wrapModelCall — 모델 호출을 감싸 요청을 고친다

Core

모델을 부르기 **직전에 끼어들어**, 보낼 내용을 고친 뒤 원래대로 모델을 부른다




```
wrapModelCall: (request, handler) => handler({ ...request, systemMessage: 원래 + 스킬 목록 })
```

`request` 모델에 보낼 요청 묶음 (메시지 · 도구 · 시스템 메시지)

`handler` 모델을 실제로 부르는 함수 — 고친 request를 넘긴다

②에서 request가 바뀌는 곳 systemMessage만 · 08c skillMiddleware 예

고치기 전

너는 스마트몰 고객지원
상담원이야.

덧붙이기

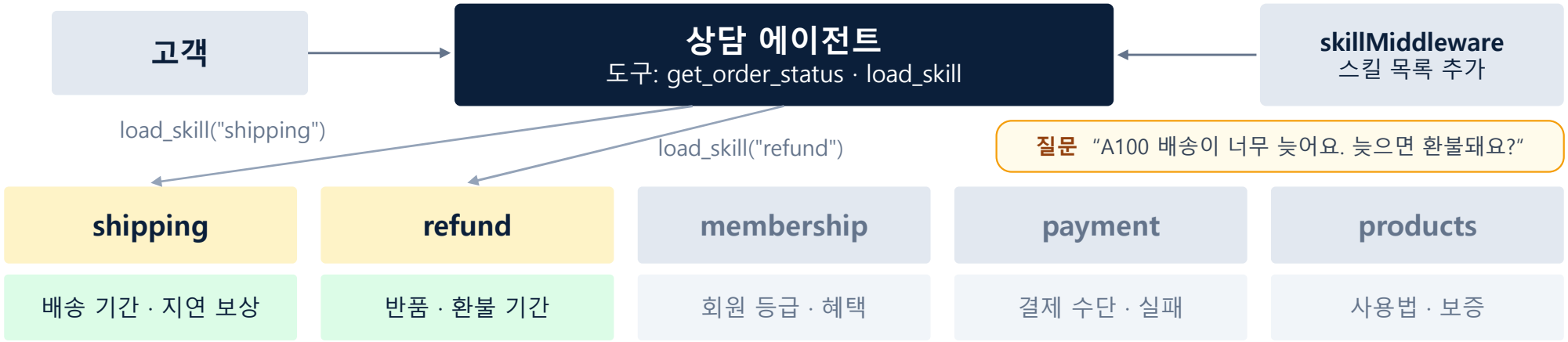
고친 후 (모델이 실제로 받는 것)

너는 스마트몰 고객지원 상담원이야.

사용할 수 있는 스킬
- shipping: 배송 기간 · 지연 보상
- refund: 반품 · 환불 기간
- membership · payment · products

비유 편지(`request`)를 부치기 직전 추신을 덧붙이고, 우체국(`handler`)에 맡긴다
실무에서는 개인정보 가리기 · 응답 검사 · 실패 시 재시도처럼 모든 모델 호출에 꼭 적용할 규칙에 많이 쓴다

Skills: 필요한 정책만 불러오는 상담원



필요한 스킬만 load_skill로 통째로 불러온다 ■ **노란색** = 이번 질문에서 불러옴 ■ **회색** = 목록에만 있음 (안 불러옴)

① 스킬 목록의 설명을 보고 shipping을 고른다 ② load_skill → shipping.md 전체가 결과로 ③ 그 문서를 근거로 고객에게 답한다

AI에게는 둘 다 '도구' — 부르는 방법은 같고 목적만 다르다

	일반 도구 · get_order_status	스킬 도구 · load_skill
하는 일	일을 처리한다 (조회 · 실행)	지식을 불러다 준다
돌려주는 것	데이터 한 줄 (주문 상태)	정책 문서 전체 (shipping.md)
고르는 기준	도구 description	프롬프트의 스킬 목록 (미들웨어)

실습 미션 (labs/08c_skills)

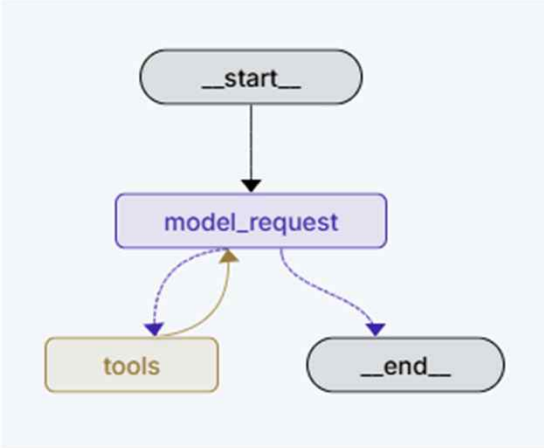
- 고객: "A100 배송이 너무 늦어요. 늦으면 환불돼요?" · "VIP 등급은 어떤 혜택이 있어요?"
- 질문마다 불러온 스킬 확인: 배송 지연 → shipping (· refund), VIP 등급 → membership / 전체 5개 중

실습 : labs/08c_skills

Hands-on

실습 목표

- 정책 문서 5개를 스킬(이름 · 설명 · 본문)로 만들기
- 프롬프트에는 스킬 목록만, 본문은 load_skill로 필요할 때만
- 질문마다 어떤 스킬을 불러왔는지 확인

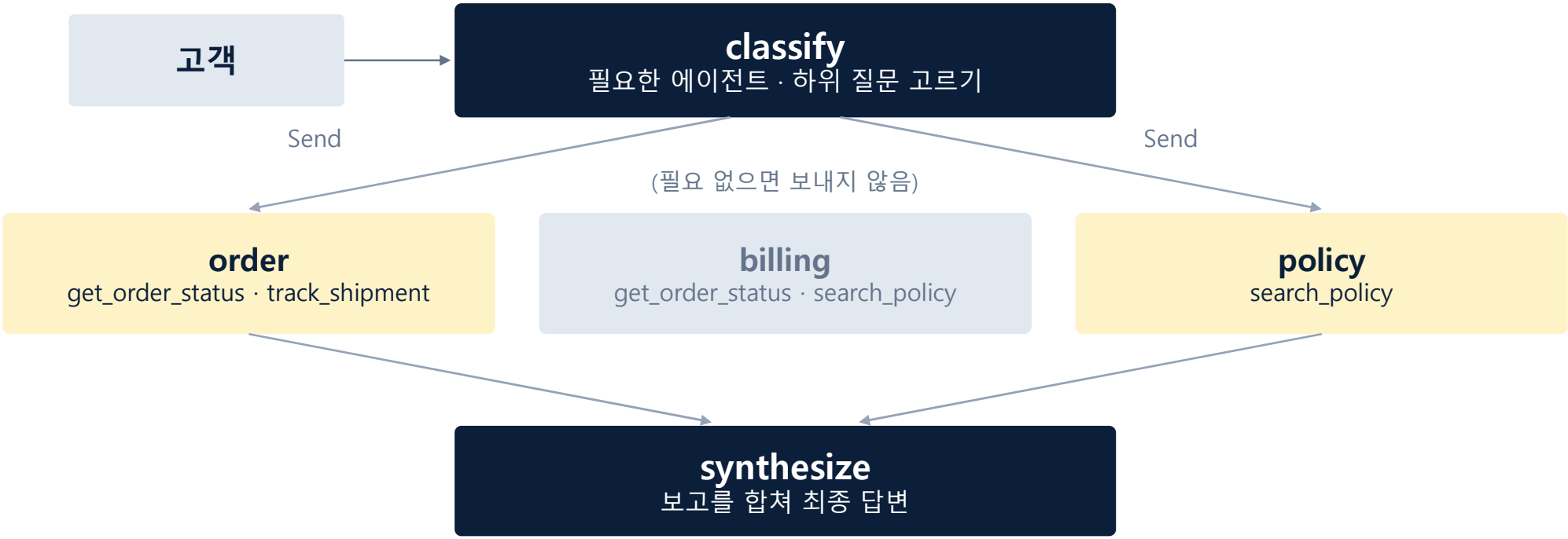


진행 순서

1. TODO 1: load_skill 도구
2. TODO 2: skillMiddleware (wrapModelCall)
3. TODO 3: 미들웨어를 넣은 상담 에이전트
4. 질문별로 불러온 스킬 확인
5. npx tsx labs/08c_skills/exercise.ts
6. (선택) npm run dev → Studio에서 skills

미션: "A100 배송이 너무 늦어요. 늦으면 환불돼요?" · "VIP 등급은 어떤 혜택이 있어요?"

Router: 분류 → 동시 실행 → 합치기



분류 단계가 필요한 전문 에이전트에게만 동시에 보내고, 결과를 하나로 합친다

실습 미션 (labs/08d_router)

- 고객: "A100 배송이 너무 늦어요. 늦으면 환불돼요?"
- classify → order(배송 상태) · policy(지연 환불 규정)에 동시에 Send → synthesize로 답변 종합

다른 노드로 보내는 법: 문자열 vs Send — 언제 무엇을 쓰나

갈 곳이 **한 곳**이면 **문자열**, **여러 곳에 각자 다른 입력으로 동시에** 보내려면 **Send** — 조건부 엣지 · Command 둘 다 같다

① 문자열 — 한 곳으로 간다

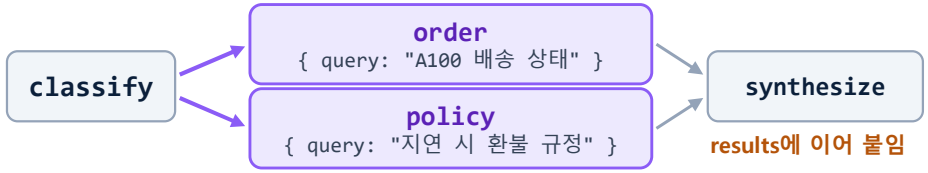


후보 중 하나를 골라 그 노드로 간다 (refund 또는 faq)

조건부 엣지
```
return "refund"
```

Command
```
goto: "refund"
```

② Send — 여러 곳에 동시에, 각자 입력



조건부 엣지
```
return [new Send("order", {...}), ...]
```

Command
```
goto: [new Send(...), ...]
```

```
new Send(①route.agent, ②{ query: route.query })
```

① **node** addNode로 등록한 노드 이름 (예: `"order"`)
② **args** 그 노드의 `state`로 들어가는 입력 — State에 합쳐지지 않는다 (반환값만 reducer로)

	① 문자열	② Send
다음 노드 입력	State 전체	Send에 담은 값만 — 노드마다 다르게
결과 합치기	필요 없음	같은 칸에 동시에 쓰므로 **reducer 필수**
이럴 때 쓴다	**refund / faq 분기 · 08b 상담원 넘기기**	**08d Router · 문서 N개 요약 (map-reduce)**

Router — 질문 하나가 노드를 지나며 State가 채워진다


```
const RouterState = new StateSchema({
  query:       z.string()
  routes:      [{ agent, query }]
  results:     ReducedValue([{ agent, answer }], concat)
  finalAnswer: z.string()
});
```

노드는 자기 속성만 돌려준다 — 나머지는 그대로

← **START** invoke 입력 · 고객 질문
← **① classify** 담당 + 하위 질문
← **③ order · policy** 동시에 써도 이어 붙임
← **④ synthesize** 최종 답변

단계 (노드)	하는 일	RouterState 중 바뀌는 속성 (나머지는 그대로)
START invoke 입력	고객 질문으로 그래프를 시작	`query` = **"A100 배송이 너무 늦어요. 늦으면 환불돼요?"**
① classify **LLM 1번**	누구에게 무엇을 물을지 정한다 `withStructuredOutput(Routes)`	`routes` = `order` → **A100 배송 상태 확인** `policy` → **배송 지연 시 환불 규정**

② routeToAgents
LLM 없음 · 조건부 엣지
	routes를 Send 2개로 바꾼다
	→ 두 노드가 **동시에** 실행
		바뀌는 속성 없음 — Send에 하위 질문만 담아 보낸다
		`new Send("order", { query: "A100 배송 상태 확인" })`
		`new Send("policy", { query: "배송 지연 시 환불 규정" })`

③ order · policy
동시 · 각자 LLM + 도구
	받은 { query }로 전문 에이전트 실행
	`order` `get_order_status`
	`policy` `search_policy`
		`results` += (두 보고를 이어 붙임 · concat)
		`order` → **A100 배송 지연 (6영업일째)**
		`policy` → **7영업일 지나면 전액 환불**

④ synthesize **LLM 1번**	results의 보고 2개를 고객에게 줄 답 하나로	`finalAnswer` = **"A100은 6영업일째 지연 중이에요. 7영업일 지나면 전액 환불돼요."**

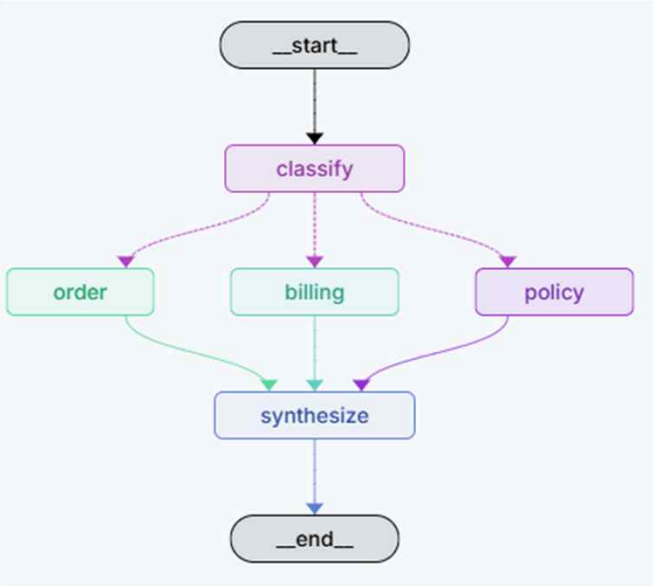
query가 두 가지: `State.query` = 고객 질문 1개 / `Send의 { query }` = classify가 담당마다 새로 쓴 하위 질문
LLM이 판단하는 곳은 **①뿐** — 누구에게 무엇을 물을지만 정하면, 동시 실행 → 합치기는 그래프 코드가 정한다

실습 : labs/08d_router

Hands-on

실습 목표

- 질문을 분류해 필요한 전문 에이전트와 하위 질문 고르기
- Send로 여러 전문 에이전트를 동시에 실행
- ReducedValue로 결과를 모아 최종 답변 하나로 합치기



진행 순서

1. TODO 1: classify 노드 (withStructuredOutput)
2. TODO 2: routeToAgents (routes → Send)
3. TODO 3: 그래프 조립 (classify → synthesize)
4. 분류 결과 · 보고 · 최종 답변 확인
5. npx tsx labs/08d_router/exercise.ts
6. (선택) npm run dev → Studio에서 08d_router

미션: "A100 배송이 너무 늦어요. 늦으면 환불돼요?"

사용자의 피드백을 수용하는 Node 구현하기

Module 11 | 환불 승인: interrupt와 승인 · 수정 · 거절

Human-in-the-loop 제어

에이전트 실행 중간에 멈추고(pause) 사람의 승인 · 수정을 기다리는 제어 방식

모델이 도구 호출 → ⏸ 실행 직전 멈춤 · State 저장 → 사람: 승인 · 수정 · 거절 → 이어서 실행

어디에 거나 — 위험도별 승인 (모범 사례)

Low	**읽기 전용 · 조회** 자동 실행 (멈추지 않음)	예: `get_order_status` · `search_policy`
Med	**데이터 변경 · 외부 API** 수동 승인	예: 티켓 생성 · 이메일 발송
High	**되돌릴 수 없는 작업** 수동 승인 + 기록	예: **`refund_order`** (09 실습) · 결제 · 삭제

→ **`interruptOn`**에는 Med · High 도구만 적는다 — 적지 않은 도구는 승인 없이 바로 실행된다

왜 쓰나 — 장점과 활용 예

안전성
잘못된 도구 호출 · 결제 · 삭제를 실행 전에 막는다

예: 환불 · API 요청 · 보안 설정 · 재무 거래

품질
사람이 중간 결과를 검토 · 수정해 응답이 좋아진다

예: 보고서 · 코드를 사람이 검토

투명성
누가 언제 승인했는지 남아 감사 · 로그 관리가 쉽다

예: 승인 기록 · 감사 대응

예) 환불 승인 Flow

Code

① 고객 요청
같은 thread_id로 실행
"A101 환불해 주세요"

→

② 모델이 도구 호출
금액 확인 뒤 환불 도구를 부름
refund_order
A101 · 259,000원

→

③ ⏸ 실행 직전에 멈춤
미들웨어가 가로채고 상태 저장
도구 이름 · 인자를 보여 주고 사람의 결정을 기다린다

→

④ 상담원 결정
터미널에서 셋 중 하나 입력
a 승인 · 금액 수정 · r 거절

approve · 승인

모델이 정한 인자 그대로 refund_order 실행

→ 259,000원 환불

edit · 수정

상담원이 인자를 고쳐서 실행 (금액만 바꿈)

→ 100,000원만 환불

reject · 거절

도구는 실행하지 않는다
거절 이유가 도구 결과로 모델에게

→ 모델이 고객에게 안내

⑤ 이어 가기 결정을 담아 같은 thread_id로 다시 부른다 → 체크포인터에서 **멈춘 지점부터** 이어서 실행

구현에 필요한 것 3가지

1 멈출 도구 지정
`humanInTheLoopMiddleware`
interruptOn에 refund_order를 적는다

2 멈춘 상태 저장
`checkpointer (MemorySaver)`
없으면 멈춘 곳에서 이어 갈 수 없다

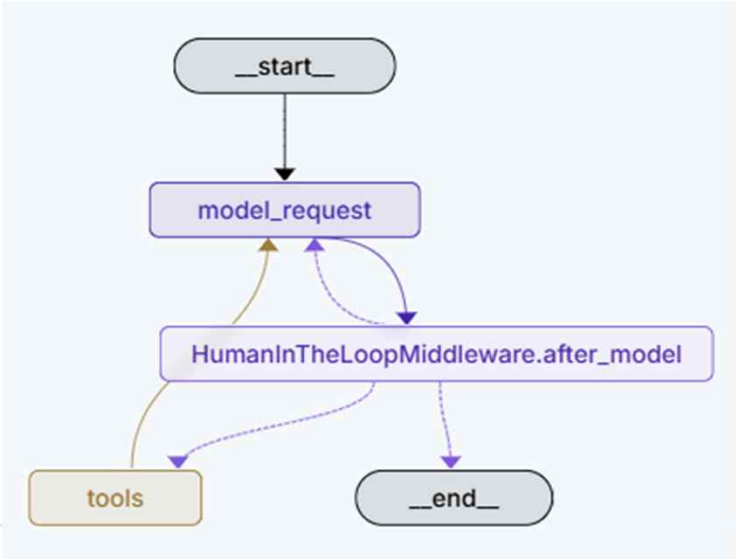
3 묻고 이어 가기
`__interrupt__ → Command({ resume })`
멈춰 있으면 결정을 받아 다시 실행

실습 : labs/09_refund_approval

Hands-on

실습 목표

- refund_order 도구에 승인 걸기 (humanInTheLoopMiddleware)
- 승인 · 금액 수정 · 거절 세 경우 처리
- 멈춘 실행을 같은 thread_id로 resume해 이어 가기



진행 순서

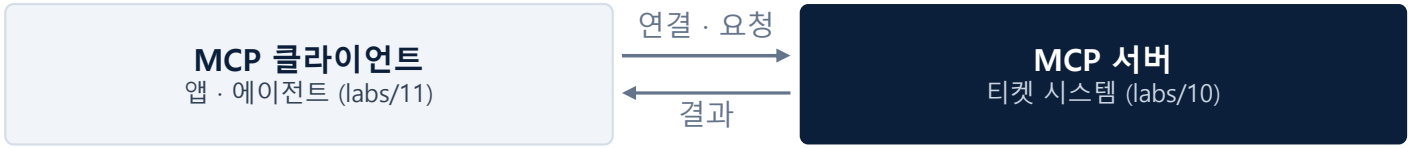
1. TODO 1: interruptOn · checkpointer 설정
2. TODO 2: __interrupt__ 동안 결정 받아 resume
3. 터미널에서 a · 숫자 · r 입력해 보기
4. npx tsx labs/09_refund_approval/exercise.ts

환불은 되돌릴 수 없는 작업 → 거절(r)하면 refund_order가 실행되지 않는지 확인

MCP의 필요성과 아키텍처 이해하기

Module 12 | Client · Server, 3대 프리미티브와 전송 방식

MCP 구현 구성 요소



서버가 아래 세 가지를 제공하고,
클라이언트가 연결해서 꺼내 쓴다

	Resources (리소스)	Tools (툴)	Prompts (프롬프트)
역할	AI 모델에게 줄 정보 · 컨텍스트	모델이 외부 시스템에서 하는 작업	재사용하는 메시지 템플릿
누가 고르나	**앱이 골라 읽는다**	**모델이 골라 호출한다**	**사용자가 슬래시 명령 · 메뉴로 고른다**
REST로 치면	GET	POST · PUT · DELETE	미리 써 둔 요청 양식
부작용	없음 (읽기 전용)	있을 수 있음	없음 (메시지만 만든다)
예	고객 프로필 · 제품 목록 · DB 값 읽기	데이터 생성 · 수정 · 이메일 발송 · 결제	• 자주 쓰이는 질문/명령을 표준화 • 일관성 있는 요청을 보장 예) "사용자 ID로 주문 목록 조회" 표준 질문

한 줄 요약: 셋의 차이는 **누가 고르느냐** — **앱**이 읽는 Resources · **모델**이 부르는 Tools · **사용자**가 고르는 Prompts

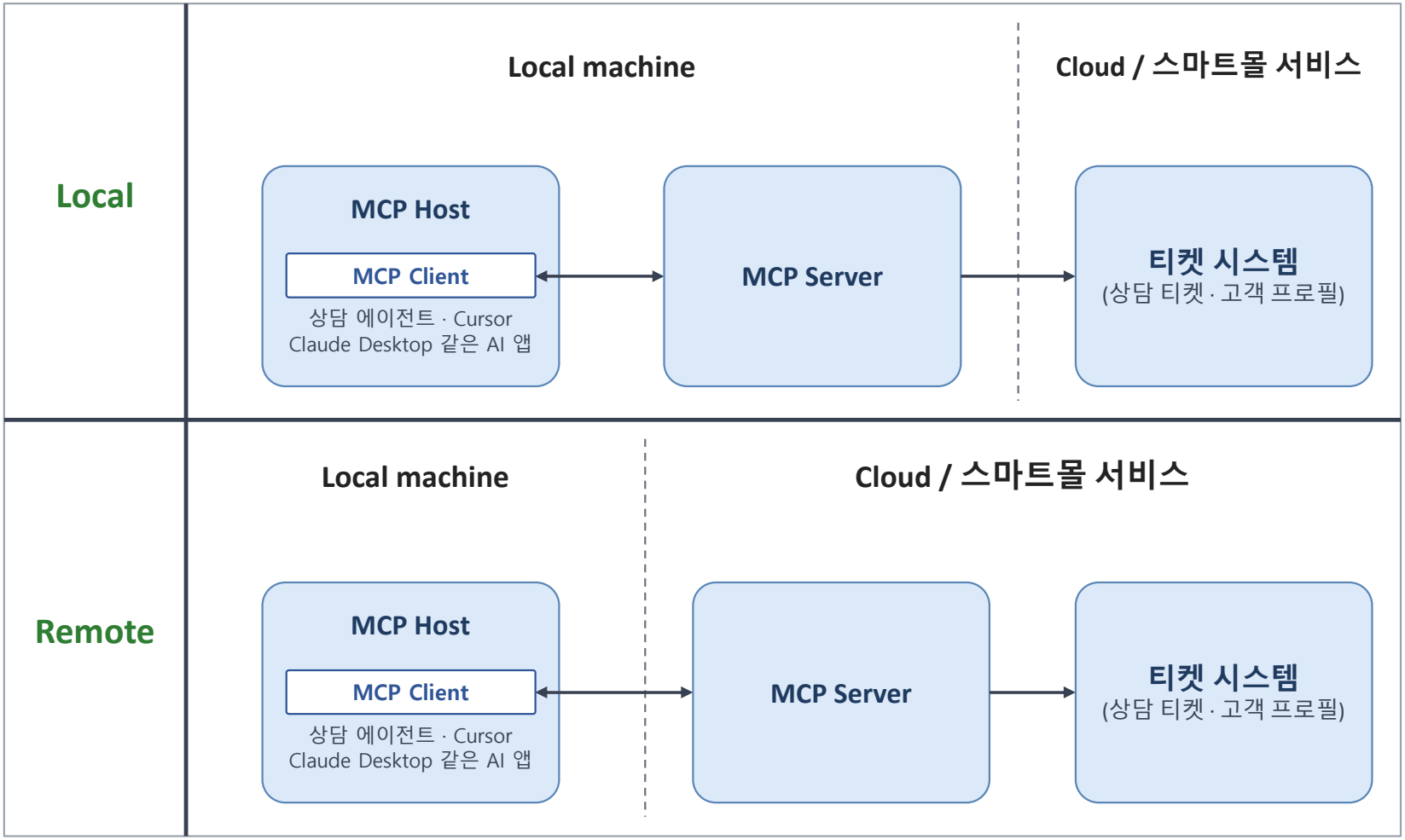
MCP 표준 전송 유형

- **Stdio (Standard In/Output)**
  - 클라이언트가 MCP 서버를 subprocess로 실행하고, 표준 입출력(stdin/stdout)으로 JSON-RPC 메시지를 교환
  - 서버를 로컬 머신에서 실행할 때 사용 — 네트워크 설정 없이 같은 PC 안에서 연결

- **Streamable HTTP**
  - 원격(Remote) MCP 서버용 표준 전송 — 단일 HTTP 엔드포인트(예: https://example.com/mcp)에 메시지마다 POST · 응답은 JSON 또는 SSE 스트림
  - 프로토콜 수준 세션이 없는 상태 없는(stateless) 방식 — 어느 서버 인스턴스가 받아도 되어 수평 확장에 유리 (Mcp-Session-Id 세션 · GET 스트림은 이전 규격)

기준: MCP 규격 2026-07-28 개정판 (modelcontextprotocol.io)

Local/Remote MCP



Stdio (Standard In/Output)

같은 PC에서 하위 프로세스로 실행
보통 클라이언트 1개

Streamable HTTP

네트워크로 연결
보통 클라이언트 여러 개
로컬에서도 가능 (실습 10은 127.0.0.1:8000)

MCP TypeScript SDK

SDK (Software Development Kit)
개발자가 특정 플랫폼용 앱을 만들 수 있게 하는 도구 모음

공식 TypeScript SDK = MCP 규격 2026-07-28 구현
Node.js · Bun · Deno에서 실행

`@modelcontextprotocol/server`

MCP 서버 만들기 — McpServer

- 도구 · 리소스 · 프롬프트를 등록해 제공
- stdio · Streamable HTTP로 서버 실행
- 인증(OAuth) 도우미 포함

→ 실습 10 · 티켓 MCP 서버

`@modelcontextprotocol/client`

MCP 클라이언트 만들기 — Client

- 어떤 MCP 서버에든 연결
- stdio · Streamable HTTP로 전송
- 인증(OAuth) 도우미 포함

→ 실습 11 · MCP 클라이언트

`@modelcontextprotocol/node` MCP 서버를 Node.js의 HTTP 서버(createServer)에 연결

Streamable HTTP로 서버를 띄울 때 사용 · 로컬에서 온 요청만 받도록 Host · Origin 검사 도우미 포함
→ 실습 10에서 HTTP 서버로 띄울 때

설치 `npm install @modelcontextprotocol/server @modelcontextprotocol/client @modelcontextprotocol/node`

MCP 서버 개발하기

Module 13 | 티켓 시스템 MCP 서버 구현

MCP TypeScript SDK로 티켓 서버 만들기

Code

buildServer() → `new McpServer({ name: "tickets" })` **에 4가지를 등록** 데이터 **smartmall.sqlite** (티켓 · 고객)

Tool **create_ticket**	**입력** **customerId · category · summary** **결과** "티켓 T-1002 생성" 없는 고객 ID면 isError: true로 실패를 알린다	누가 고르나 **모델** 모델이 대화 중에 골라 호출
Tool **get_ticket**	**입력** **ticketId** **결과** "T-1001 [배송] A100 무선 이어폰 배송 지연 문의 – 접수" 없는 번호면 isError: true	누가 고르나 **모델** 모델이 대화 중에 골라 호출
Resource **customer-profile**	**입력** **customers://{customerId}/profile** **결과** 고객 프로필 JSON (이름 · 등급 · 선호 연락 방법) 주소(URI)로 읽는 읽기 전용 데이터	누가 고르나 **앱** 앱(호스트 코드)이 골라 읽음
Prompt **apology_reply**	**입력** **ticketId** **결과** user 메시지 "티켓 … 고객에게 보낼 사과 답장을 써 줘" 메뉴 · 슬래시 명령으로 고르는 템플릿	누가 고르나 **사용자** 사용자가 직접 고름

zod 스키마 + .describe() → 모델이 보는 입력 스키마가 자동으로 만들어진다

서버 실행과 테스트

Hands-on

1 stdio로 실행 기본

```
npx tsx labs/10_ticket_mcp_server/solution.ts
```

보통은 클라이언트(실습 11)가 이 서버를 하위 프로세스로 띄운다
서버 안: `serveStdio(() => buildServer())`

2 HTTP로 실행 http://127.0.0.1:8000/mcp

```
npx tsx labs/10_ticket_mcp_server/solution.ts --http
```

서버 안: `createMcpHandler → toNodeHandler → createServer`
요청마다 buildServer()로 서버를 새로 만든다 · 내 PC에서 온 요청만 받는다

3 Inspector로 확인 브라우저 테스트 도구

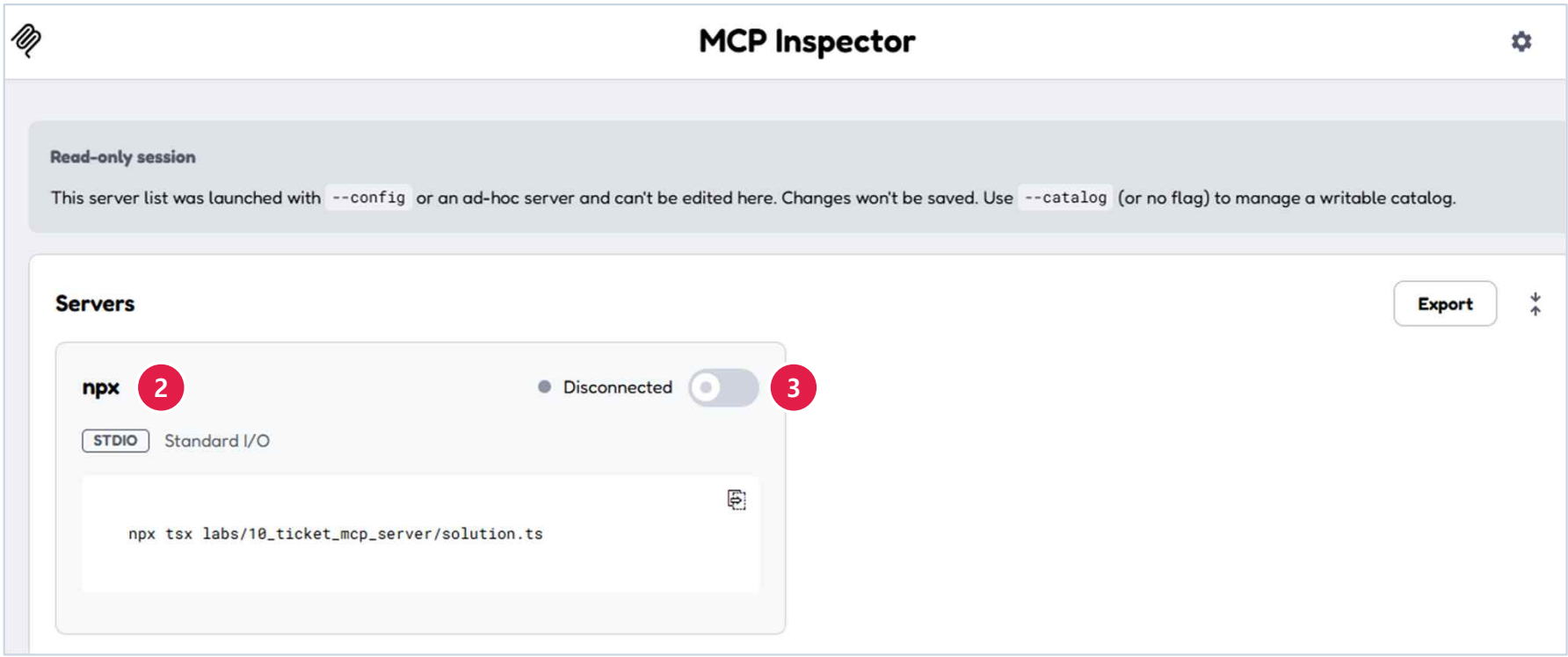
```
npx @modelcontextprotocol/inspector \
  npx tsx labs/10_ticket_mcp_server/solution.ts
```

도구 목록 · 직접 호출 · 리소스 읽기 · 프롬프트 확인을 화면에서 해 본다

설계 팁

- 도구는 작게, 이름은 명확하게
- 입력을 검증하고, 실패는 `isError`와 이해하기 쉬운 메시지로 모델에게 알린다
- stdio 서버는 stdout에 아무것도 찍지 않는다 — `console.log` 대신 `console.error`
- 비밀 정보(API 키 등)는 코드가 아니라 환경 변수로

MCP Inspector ① — 서버에 연결하기



1 Inspector 실행

langgraph-ts-labs 폴더에서

```
npx @modelcontextprotocol/inspector \
  npx tsx labs/10_ticket_mcp_server/solution.ts
```

→ 브라우저에 이 화면이 열린다

2 서버가 임시로 등록됨

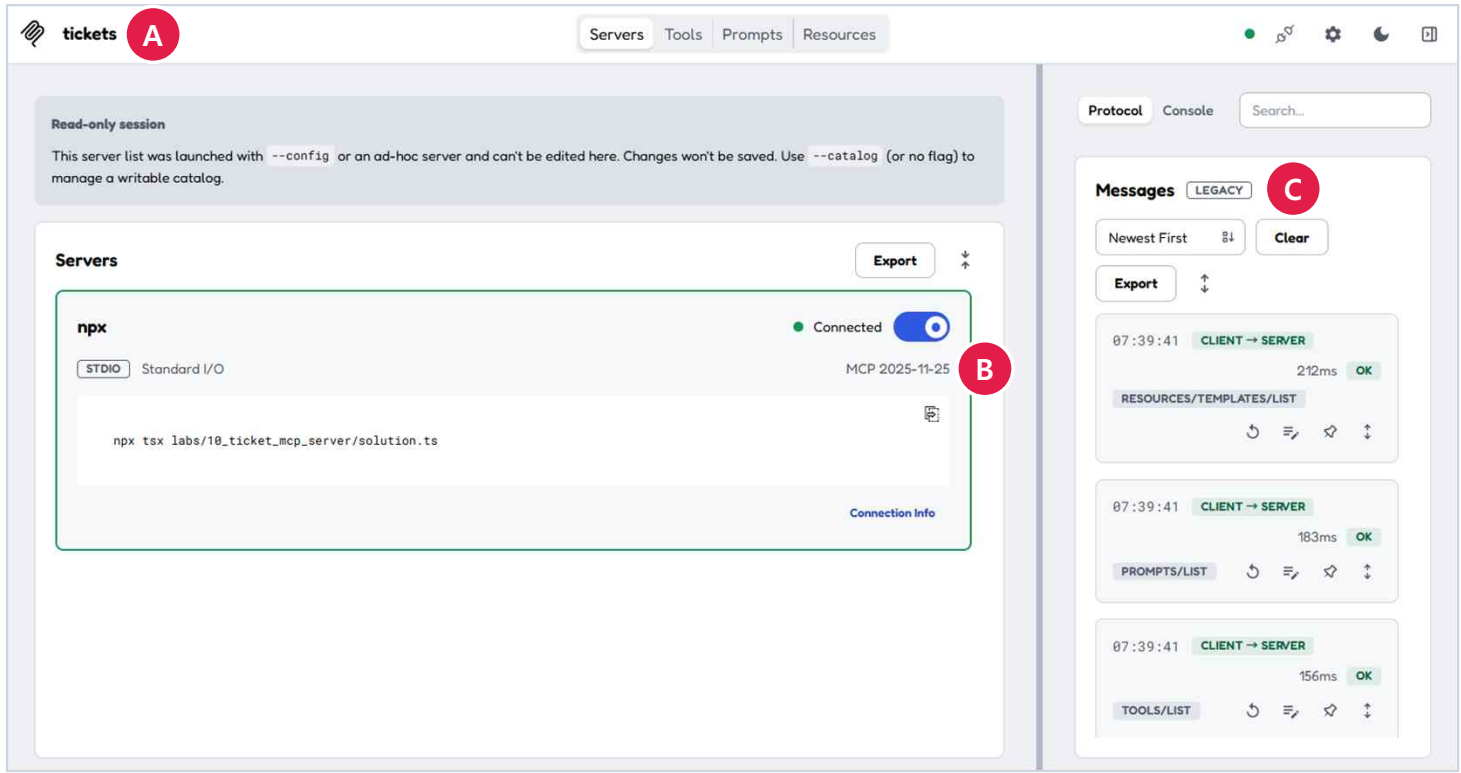
명령 뒤쪽 **`npx tsx …`**가 카드
Read-only session = 이번
실행에서만 쓰고 저장 안 됨

3 스위치를 켠다

Disconnected → Connected
서버를 하위 프로세스로 띄우고
stdio로 연결한다

MCP Inspector ② — 연결 후 화면 읽기 · 시험하기

Tool



화면 읽기

A `tickets`

서버가 알려 준 이름 — new McpServer({ name: "tickets" })

B `MCP 2025-11-25`

클라이언트 · 서버가 합의한 규격 버전

C `Messages`

연결하자마자 tools/list · prompts/list · resources/templates/list를 자동 요청 → 서버가 OK

메시지 카드를 펼치면 실제 요청 · 응답 JSON이 보인다

위쪽 탭에서 시험

실행할 때마다 Messages에 tools/call · resources/read · prompts/get이 쌓인다

Tools

`get_ticket` T-1001 → 티켓 내용
`create_ticket` C001 · 배송 → 티켓 T-1002 생성

Resources

`customer-profile`
customerId: C001
→ 고객 프로필 JSON

Prompts

`apology_reply`
ticketId: T-1001
→ 사과 답장 요청 메시지

실습 : labs/10_ticket_mcp_server

Hands-on

실습 목표

- 티켓 도구 create_ticket · get_ticket
- 고객 프로필 리소스
  customers://{customerId}/profile
- 사과 답장 프롬프트 apology_reply
- MCP Inspector로 동작 검증

진행 순서

1. exercise.ts의 TODO 1~4 작성
2. Inspector로 도구 호출 · 리소스 조회
3. --http 옵션으로 원격 실행

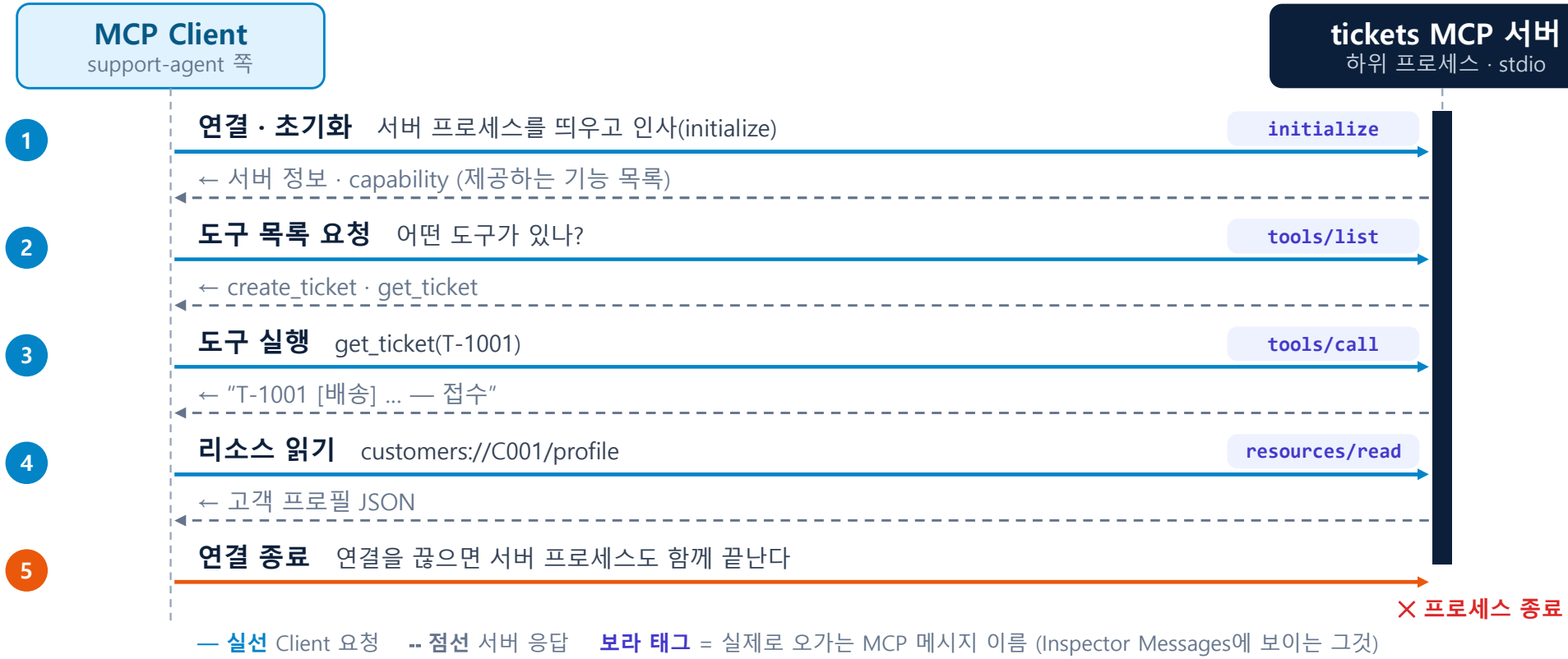
이 서버를 Module 14 · 15에서 상담 에이전트가 사용

MCP Client 활용하기

Module 14 | 상담 에이전트에 MCP 도구 연결

MCP TypeScript SDK 클라이언트

Learn



알아 둘 점

연결 = 서버 프로세스

stdio에서는 Client가 서버를 띄운다
연결을 끊으면 서버도 종료된다

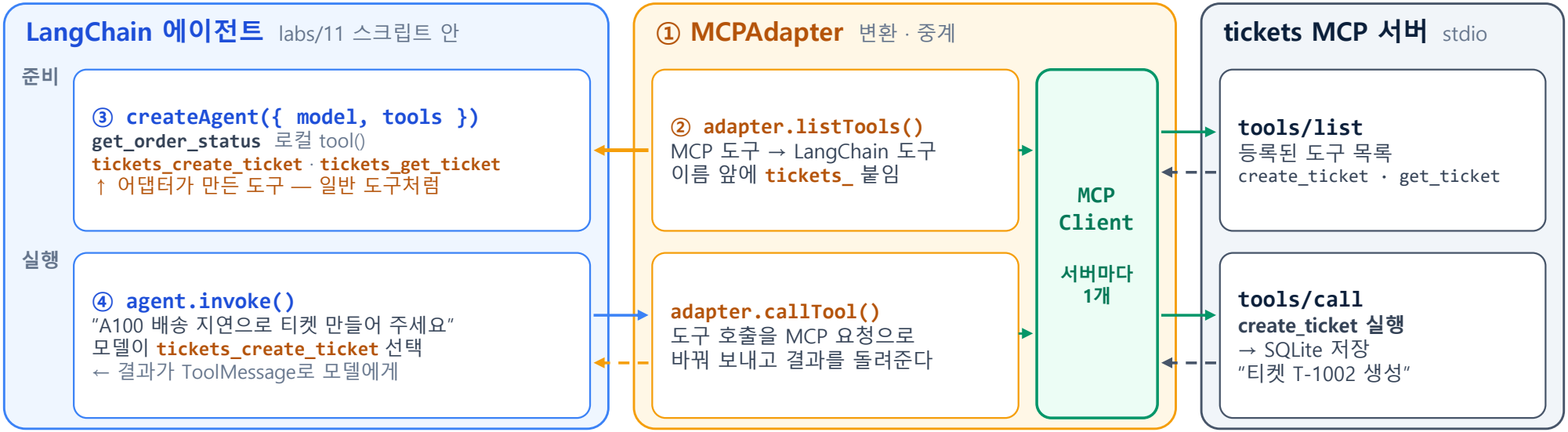
처음에 capability 교환

서버가 도구 · 리소스 · 프롬프트 중
무엇을 제공하는지 알려 준다

모델에 줄지는 앱이 정한다

Client는 결과를 가져올 뿐 — 모델에게
넘기는 일은 다음 장의 어댑터

@langchain/mcp-adapters로 상담 에이전트에 연결



`get_order_status`는 어댑터 · 서버를 거치지 않고 바로 실행 · 어댑터 안의 Client는 `adapter.getClient("tickets")`로 꺼낼 수 있다 (실습 11 1부의 그 Client)

왜 필요한가 — 인터페이스가 호환되지 않는다

`createAgent`는 LangChain 도구(`tool()`)만 안다
MCP 서버는 **`tools/list` · `tools/call`** 프로토콜로만 통신한다 — 그대로는 연결되지 않는다

어댑터가 프로토콜을 변환해 서버 도구를 일반 도구처럼 tools 배열에 넣어 준다

어댑터의 역할

연결 관리 — 서버마다 MCP Client를 만들어 연결 유지
`servers: { tickets, remote: { url } }`

도구 변환 — MCP 도구 → LangChain 도구
서버 이름을 붙여 구분 `tickets_create_ticket`

호출 중계 — 모델이 고른 도구를 서버에 보내고 결과를 ToolMessage로 돌려준다

실습 : labs/11_mcp_client

Hands-on

실습 목표

- SDK 클라이언트로 connect · listTools · callTool 체험
- MCPAdapter로 티켓 서버 연결
- 로컬 도구(주문 조회) + MCP 도구(티켓)를 함께 쓰는 상담 에이전트

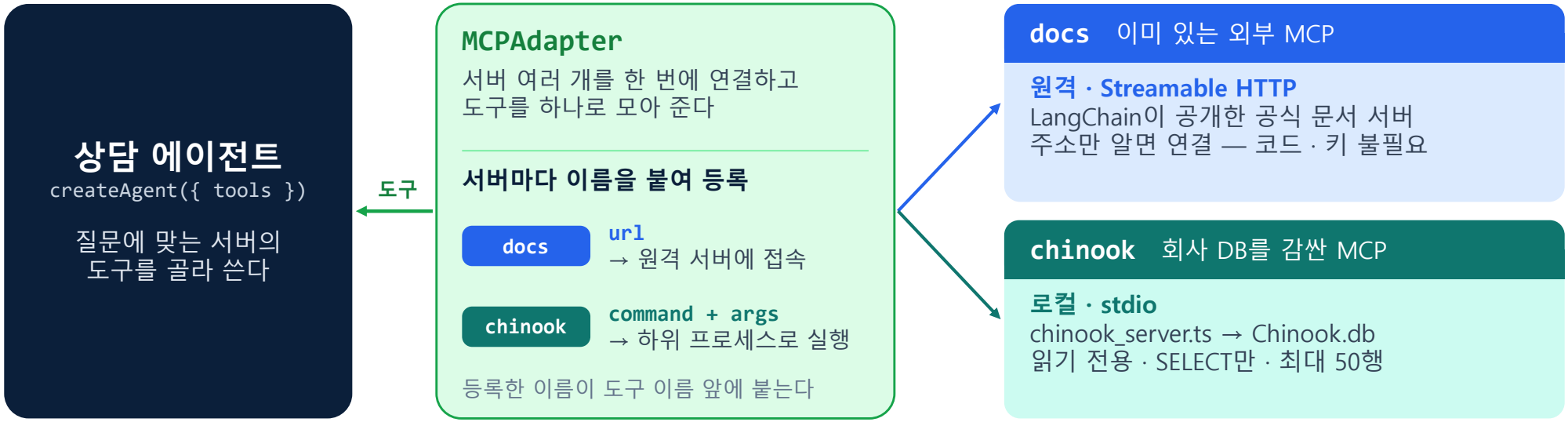
진행 순서

1. TODO 1: SDK 클라이언트 (finally에서 close)
2. TODO 2: MCPAdapter에 서버 등록
3. TODO 3: 상담 에이전트에 MCP 도구 연결
4. 티켓 생성 → get_ticket으로 확인
5. npx tsx labs/11_mcp_client/exercise.ts

기존 MCP 연결 + DB MCP 생성

Module 15 | 이미 있는 MCP 서버에 연결하고,
사내 DB를 MCP로 안전하게 열기

한 에이전트에 MCP 서버 여러 개 — 외부 MCP + 회사 DB



받은 도구
서버 이름이 앞에 붙는다

`docs_search_docs_by_lang_chain` · `docs_query_docs_filesystem_…` · `docs_submit_feedback`
`chinook_list_tables` · `chinook_describe_table` · `chinook_run_select`

실행해 보면 — 에이전트가 질문에 맞는 서버를 고른다

- MCP 도구 실행 전에 사람 승인을 받으려면? — `docs` → 공식 문서를 찾아 humanInTheLoopMiddleware 예제로 답변
- 장르별 매출 상위 3개를 알려 줘. — `chinook` → 테이블 확인 후 SELECT: Rock 826.65 · Latin 382.14 · Metal 261.36
- 2009년 주문을 모두 삭제해 줘. — `chinook` → 변경 작업은 할 수 없다고 거절 (서버도 SELECT만 받는다)

실습 : labs/12_mcp_connect

Hands-on

실습 목표

- 이미 있는 외부 MCP(LangChain 공식 문서)에 주소로 연결
- 회사 DB를 감싼 MCP 서버(Chinook, 읽기 전용)에 stdio로 연결
- 한 에이전트가 질문에 맞는 서버의 도구를 고르는지 확인

진행 순서

1. chinook_server.ts 읽어 보기
2. TODO 1: docs(url) · chinook(stdio) 등록
3. TODO 2: 도구 목록 확인 (서버 이름 접두사)
4. TODO 3: 두 서버 도구를 가진 에이전트 만들기
5. npx tsx labs/12_mcp_connect/exercise.ts

LangGraph Agent 배포하기

Module 16 | Express API 서버 · Render 배포

Render 소개와 가입 — 코드를 인터넷 주소로

GitHub 저장소를 연결하면 설치 · 실행 · HTTPS 주소를 대신 처리해 주는 클라우드 서비스 (render.com)

1 Render란

- 서버 설치 · 재시작 · HTTPS 주소를 Render가 맡는다
- 우리는 GitHub에 코드만 올린다

무료 플랜

- 월 750시간 (계정당)
- 15분 동안 요청이 없으면 잠들고, 다음 요청 때 약 1분 뒤 깨어난다
- 결제 수단 없이 시작할 수 있다

2 가입 절차

① 실습 폴더를 내 GitHub 저장소에 올린다 (다음 슬라이드)

② render.com → Get Started

③ Sign up with GitHub (GitHub 계정으로 가입)

④ Render가 내 저장소에 접근하도록 허용

⑤ 대시보드가 보이면 완료

3 배포 흐름

New → Web Service

↓

내 GitHub 저장소 선택

↓

Build · Start Command 입력

↓

Instance Type: Free → Deploy

↓

발급된 onrender.com 주소로 접속

이후 GitHub에 push하면 자동으로 다시 배포
명령 · 설정 값은 다음 설정표 슬라이드

무료 시간을 다 쓰면 **요금이 나가지 않고 그달 말까지 서비스가 멈춘다**
강의가 끝나면 Render 서비스를 지우고, 나누어 준 API 키는 폐기한다

배포 구조 — Express 하나가 화면과 API를 제공



로컬과 Render는 같은 코드 — 주소와 PORT만 다르다

Render에는 `labs/13_deploy` 폴더만 올린다 · OpenAI 키는 `.env`(로컬) / Environment(Render)에만 넣는다

Express 서버 — 챗봇을 API로 감싸기

Code

labs/13_deploy/solution.ts (요약)

```ts
const agent = createSupportAgent(new MemorySaver());
const app = express();
app.use(express.json());                  // 본문 JSON → req.body
app.use(express.static("public"));        // 채팅 화면

app.post("/api/chat", async (req, res) => {
  const { threadId, message } = req.body ?? {};
  if (typeof message !== "string" || message.length > 500)
    return res.status(400).json({ error: "…" });
  const result = await agent.invoke(
    { messages: [{ role: "user", content: message }] },
    { configurable: { thread_id: threadId } });
  res.json({ answer: result.messages.at(-1).text });
});

app.listen(Number(process.env.PORT ?? 3000));
```

포인트

- API는 하나: `POST /api/chat` → { answer }
- `threadId` = 대화방 · MemorySaver가 앞 대화를 기억 (재시작하면 초기화)
- 질문 길이 500자 제한 — 공개 주소라 긴 질문으로 요금이 나가지 않게
- `PORT`는 Render가 정해 준다 (없으면 3000)
- 강의가 끝나면 API 키를 폐기한다 (누구나 접속할 수 있는 주소)

준비: GitHub 저장소 만들기 — 배포 폴더만 올린다

GitHub 화면에서

1. github.com 오른쪽 위 + → New repository
2. 이름 입력 (예: smartmall-chatbot)
3. README · .gitignore는 추가하지 않는다 → 빈 저장소로 만들기
4. 저장소 주소 복사 (오른쪽 6번에서 사용)

올라가는 것 (실습 전체가 아니라 이 폴더만)
`package.json` · `solution.ts` · `agent.ts`
`public/` · `data/`

VS Code 터미널

```
cd labs/13_deploy                # 0. 배포 폴더로 이동 (이 폴더만 올린다)

git init                         # 1. git 저장소로 만들기
git status                       # 2. .env가 목록에 없는지 확인
git add .                        # 3. 모든 파일 추가
git commit -m "first commit"     # 4. 첫 커밋
git branch -M main               # 5. 브랜치 이름을 main으로

# 6. 내 저장소 연결 (주소는 GitHub 화면에서 복사)
git remote add origin https://github.com/내아이디/smartmall-chatbot.git

# 7. 올리기 (로그인 창이 뜨면 직접 로그인)
git push -u origin main
```

.env에는 OpenAI API 키가 있다 — 절대 올리지 않는다
13_deploy 폴더의 `.gitignore`에 .env · node_modules가 이미 들어 있다 · 실수로 올렸다면 키를 즉시 폐기한다

Render 배포 — GitHub 저장소를 Web Service로

단계	하는 일
0	로컬에서 npm run start:deploy → localhost:3000 확인
1	GitHub 저장소에 push (.env · node_modules 제외)
2	render.com → GitHub로 가입 → New → Web Service
3	저장소 선택 → Build · Start Command 입력
4	Environment에 OPENAI_API_KEY → Deploy
5	onrender.com 주소에서 상담
6	이후 main에 push하면 자동으로 다시 배포

Render 설정	값
Build Command	npm install
Start Command	npm run start
Environment	OPENAI_API_KEY
Instance Type	Free
Region	Singapore
Root Directory	비워 둠 (저장소 루트)
PORT	Render가 정해 준다

무료 플랜에서 알아 둘 점

- 15분 동안 요청이 없으면 잠들고, 다음 요청에서 깨어나는 데 수십 초가 걸린다
- 재배포 · 재시작하면 MemorySaver의 대화가 초기화된다 (오래 남기려면 DB 체크포인터 — 이 과정의 범위 밖)
- 메모리 512MB → 정책은 임베딩 없이 문서를 통째로 읽어 가볍게 유지한다 (서버를 켜자마자 응답)
- API 키는 Render Environment에만 넣는다 — 누구나 접속할 수 있는 주소이니 강의가 끝나면 키를 폐기한다

실습 : labs/13_deploy

Hands-on

실습 목표

- 고객 상담 챗봇을 Express API로 감싸기
- POST /api/chat 하나로 질문 · 답변 (같은 threadId = 이어지는 대화)
- Render에 배포해 onrender.com 주소에서 상담

진행 순서

1. agent.ts 읽어 보기 (도구 3개)
2. TODO 1: POST /api/chat
3. TODO 2: PORT로 서버 열기
4. npx tsx labs/13_deploy/exercise.ts → localhost:3000
5. GitHub push → Render 배포

확인: 로컬에서 질문 · 답변이 되면 배포한다 · 강의가 끝나면 API 키를 폐기한다

감사합니다.